

FDU 数值算法 4. Krylov 子空间方法

本文根据邵老师授课内容整理而成，并参考了以下教材：

- 数值线性代数 (第二版, 徐树方, 高立, 张平文) 第 5 章
- Applied Numerical Linear Algebra (J. Demmel) Chapter 6
- 应用数值线性代数 (J. Demmel) 第 6 章
- Iterative Methods for Sparse Linear Systems (2nd Edition, Y. Saad) Chapter 6, 7
- Matrix Computation (4th Edition, G. Golub & C. Van Loan) Chapter 9, 10

欢迎批评指正！

4.1 线性方程组

4.1.1 基本框架

首先考虑使用 Krylov 子空间方法求解线性方程组 $Ax = b$

我们默认 $A \in \mathbb{C}^{n \times n}$ 是大型稀疏矩阵 (毕竟小型稠密矩阵可以使用前面介绍的直接法求解)

简单起见, 假设 A 是非奇异的, 我们的目标便是 (近似) 计算 $x_* = A^{-1}b$

根据 Cayley-Hamilton 定理可知 A^{-1} 可以表示为 A 的不超过 $n-1$ 阶的多项式.

因此 $x_* = A^{-1}b$ 一定落在 $\text{span}\{b, Ab, \dots, A^{n-1}b\}$ 中.

(Cayley-Hamilton 定理, Matrix Analysis 定理 2.4.3.2)

设 $p_A(t) := \det(tI_n - A)$ 是 $A \in \mathbb{C}^{n \times n}$ 的特征多项式, 则我们有 $p_A(A) = 0_{n \times n}$ 成立.

换言之, 任意复方阵都满足其特征方程.

基于 Cayley-Hamilton 定理可将非奇异阵 $A \in \mathbb{C}^{n \times n}$ 的负幂 A^k ($k \leq -1$) 写成 $I, A, \dots, A^{n-1}$ 的线性组合.

(Matrix Analysis 推论 2.4.3.4)

设 $A \in \mathbb{C}^{n \times n}$ 非奇异

记其特征多项式 $p_A(t) = \det(tI - A) = t^n + c_{n-1}t^{n-1} + \dots + c_1t + c_0$

根据 Cayley-Hamilton 定理我们有 $p_A(A) = A^n + c_{n-1}A^{n-1} + \dots + c_1A + c_0I_n = 0_{n \times n}$

则 $A^{-1} = -\frac{1}{c_0}(A^{n-1} + c_{n-1}A^{n-2} + \dots + c_2A + c_1I)$

定义关于 A, b 的 k 阶 Krylov 子空间:

$$\mathcal{K}(A, b, k) := \text{span}\{b, Ab, \dots, A^{k-1}b\} \quad (k = 1, \dots, n)$$

记 x_k 为 $x_* = A^{-1}b$ 在 $\mathcal{K}(A, b, k)$ 中的某种意义下的最佳近似.

(无论如何定义最佳近似, 我们显然要保证 $x_n = x_* = A^{-1}b \in \mathcal{K}(A, b, n)$ 成立)

注意到:

$$\mathcal{K}(A, b, 1) \subseteq \mathcal{K}(A, b, 2) \subseteq \dots \subseteq \mathcal{K}(A, b, n)$$

因此近似解 x_k 对精确解 $x_* = A^{-1}b$ 的近似效果会随着 k 的增大而 (非严格地) 单调变好.

4.1.2 基本分类

现在我们需要定义何为 $x_* = A^{-1}b$ 在 k 阶 Krylov 子空间 $\mathcal{K}(A, b, k)$ 的最佳近似 x_k

不同的定义 (如果可以定解的话) 将发展出不同的算法.

下面是几种自然的定义:

- ① 极小化误差:

$$x_k := \arg \min_{x \in \mathcal{K}(A, b, k)} \|x_* - x\|_2$$

遗憾的是, Krylov 子空间没有足够的信息去求解上述优化问题.

- ② 极小化残量:

$$x_k := \arg \min_{x \in \mathcal{K}(A, b, k)} \|b - Ax\|_2$$

注意到 $\|b - Ax\|_2$ 在 $\mathbb{C}^n$ 的子空间 $\mathcal{K}(A, b, k)$ 上连续且下有界, 故上述优化问题是存在全局最优解。

(趋于远端时函数值趋于正无穷, 因而可以将其限制在紧集上应用 Weierstrass 定理)

当 A 是对称阵时, 相应的算法称为**极小化残量法** (MINRES)

当 A 是非对称矩阵时, 相应的算法称为**广义极小残量法** (GMRES) (前者发现的更早, 故排在前面)

(另一个存活的算法是 BICGSTAB)

- ③ 残量正交化:

$$\begin{aligned} x_k &= x \in \mathcal{K}(A, b, k) \text{ such that } (b - Ax) \perp \mathcal{K}(A, b, k) \\ &= \text{projection of } b \in \mathbb{C}^n \text{ on } \mathcal{K}(A, b, k) \\ &= x \in \mathcal{K}(A, b, k) \text{ such that } Q_k^H(b - Ax) = 0_k \end{aligned}$$

其中 Q_k 是由 Arnoldi 过程 (或 Lanczos 过程, 如果 A 是实对称阵的话) 得到的 $\mathcal{K}(A, b, k)$ 的一组标准正交基 (构成的矩阵)

简单起见, 我们假设 $b \notin \mathcal{K}(A, b, k)$, 故有 $Q_k = [q_1, \dots, q_k] \in \mathbb{C}^{n \times k}$

当 A 是非对称矩阵时, 相应的算法为**全正交法** (Fully Orthogonal Method, FOM) (已淘汰)

当 A 是对称阵时, 相应的算法为 **Lanczos 正交法** (Symmetric Lanczos Quadrature, SYMMLQ)

当 A 对称正定时, 残量正交化准则与以下准则等价:

$$\begin{aligned} x_k &:= \arg \min_{x \in \mathcal{K}(A, b, k)} (x_* - x)^T A (x_* - x) \quad (\text{note that } x_* = A^{-1}b) \\ &= \arg \min_{x \in \mathcal{K}(A, b, k)} \|x_* - x\|_A \quad (\text{where } \|z\|_A := \langle z, z \rangle_A^{\frac{1}{2}} = \sqrt{z^T A z}) \\ &= \arg \min_{x \in \mathcal{K}(A, b, k)} r^T A^{-1} r \quad (\text{denote } r := b - Ax) \\ &= \arg \min_{x \in \mathcal{K}(A, b, k)} \|r\|_{A^{-1}} \quad (\text{where } \|z\|_{A^{-1}} := \langle z, z \rangle_{A^{-1}}^{\frac{1}{2}} = \sqrt{z^T A^{-1} z}) \end{aligned}$$

相应的算法为**共轭梯度法** (Conjugate Gradient Method, CG)

上述算法的稳定性通常依赖于 $\kappa(A)$

若非对称线性系统 $Ax = b$ 是良态的 (即 $\kappa(A)$ 比较小),

则我们可以将其转化为对称正定线性系统 $A^T A x = A^T b$ 并使用共轭梯度法求解 (通常会比 GMRES 代价更小)

这称为**正则化方法**, 其稳定性依赖于 $(\kappa(A))^2$

4.1.3 重启动

考虑大型稀疏线性系统 $Ax = b$ (其中 $A \in \mathbb{C}^{n \times n}$, 而 n 至少是百万级别)

注意到保存 k 阶 Krylov 子空间 $\mathcal{K}(A, b, k)$ 的标准正交基 $Q_k \in \mathbb{C}^{n \times k}$ 的存储复杂度为 $O(nk)$

同时 Krylov 子空间方法的计算复杂度通常与 k 或 k^2 正相关。

因此当 k 达到一定量级后, 可能出现内存不够的问题,

此时我们需要进行重启动 (效果不一定好, 这只是没有办法的办法)

具体来说, 记解的初值为 $x^{(0)} \in \mathbb{C}^n$, 初始残量为 $r^{(0)} := b - Ax^{(0)}$

于是线性系统 $Ax = b$ 变为 $A(x - x^{(0)}) = b - Ax^{(0)} = r^{(0)}$

根据 $A, r^{(0)}$ 生成 k 阶 Krylov 子空间 $\mathcal{K}(A, r^{(0)}, k)$

此时寻找 x_* 在 $\mathcal{K}(A, b, k)$ 中的最佳近似 $x^{(k)}$ 就等价于寻找 $x_* - x^{(0)}$ 在 $\mathcal{K}(A, r^{(0)}, k)$ 中的最佳近似:

$$x^{(k)} \in \mathcal{K}(A, b, k) = \{x^{(0)} + \mathcal{K}(A, r^{(0)}, k)\}$$

当 k 达到一定量级后, 我们将近似解 $x^{(k)}$ 作为初值重新进行 Krylov 子空间迭代 (即重启动)

4.1.4 预条件

粗略来说, 线性系统 $Ax = b$ 的预优矩阵 $M \in \mathbb{C}^{n \times n}$ 需要满足以下条件:

- ① M 是稀疏的
- ② M 是非奇异的, 且 $M \approx A$
这可以用 $\kappa(M^{-1}A) \ll \kappa(A)$ 来刻画 (通常来说, $\kappa(M^{-1}A)$ 越趋近于 1 越好)
- ③ 形如 $Mz = r$ 的方程组易于求解 (例如三角方程组)
换言之, M 容易求逆: 对 M 求多次逆的代价要小于对 A 求一次逆的代价

此时线性方程组 $Ax = b$ 具有以下等价表示:

$$\begin{aligned}
Ax &= b \\
&\Leftrightarrow \\
(AM^{-1})(Mx) &= b \\
&\Leftrightarrow \\
(M^{-1}A)X &= M^{-1}b \\
&\Leftrightarrow \\
(M^{-1}AM^{-1})(Mx) &= M^{-1}b
\end{aligned}$$

分别称为右预条件、左预条件和双预条件.

其中 AM^{-1} 和 $M^{-1}A$ 无需显式计算.

因为我们只需做稀疏矩阵-向量乘法, $z = AM^{-1}x$ 的乘法只需拆分为 $y = M^{-1}x$ 和 $z = Ay$ 两次乘法即可.

常用的预优矩阵如下:

- ① 对角预优矩阵:

若系数矩阵 A 的对角元相差较大, 则可取 $M = \text{diag}(a_{11}, \dots, a_{nn})$

若系数矩阵 $A = \begin{bmatrix} A_{11} & \cdots & A_{1p} \\ \vdots & & \vdots \\ A_{p1} & \cdots & A_{pp} \end{bmatrix}$ 的对角块 A_{ii} ($i = 1, \dots, p$) 是易于求逆的方阵,

则可取 $M = \text{diag}(A_{11}, \dots, A_{pp})$

- ② 不完全三角分解:

(对称正定时使用不完全 Cholesky 分解, 待补充, Matrix Computation 10.3 节)

部分填入以保证稀疏性.

每步 Krylov 子空间迭代的预条件甚至都可以不一样.

4.2 GMRES

4.2.1 基础算法

给定 $A \in \mathbb{R}^{n \times n}$ 和 $b \in \mathbb{R}^n$

设初始向量为 $x_0 \in \mathbb{R}^n$, 记初始残差向量为 $r_0 := b - Ax_0$

假设已经通过 Arnoldi 过程得到了 k 阶 Krylov 子空间 $\mathcal{K}(A, r_0^{(0)}, x)$ 的一组标准正交基 $Q_k = [q_1, \dots, q_k]$, 满足:

$$\begin{aligned}
AQ_k &= A[q_1, \dots, q_k] \\
&= [q_1, \dots, q_k, q_{k+1}] \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1k} \\ h_{21} & h_{22} & \cdots & h_{2k} \\ & \ddots & \ddots & \vdots \\ & & h_{k,k-1} & h_{k,k} \\ & & & h_{k+1,k} \end{bmatrix} \\
&= Q_{k+1} \tilde{H}_k \\
&= [q_1, \dots, q_k] \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1k} \\ h_{21} & h_{22} & \cdots & h_{2k} \\ & \ddots & \ddots & \vdots \\ & & h_{k,k-1} & h_{k,k} \end{bmatrix} + q_{k+1} h_{k+1,k} \\
&= Q_k H_k + \text{rank-one} \quad (\text{where rank-one} := q_{k+1} h_{k+1,k})
\end{aligned}$$

我们的目标是将残量最小化:

$$\begin{aligned}
x_k &:= \arg \min_{x \in x_0 + \mathcal{K}(A, r_0, k)} \|b - Ax\|_2 \quad (\text{denote } x = x_0 + Q_k y \text{ where } y \in \mathbb{R}^k) \\
&= x_0 + Q_k \cdot \arg \min_{y \in \mathbb{R}^k} \|b - A(x_0 + Q_k y)\|_2 \\
&= x_0 + Q_k \cdot \arg \min_{y \in \mathbb{R}^k} \|r_0 - AQ_k y\|_2 \\
&= x_0 + Q_k \cdot \arg \min_{y \in \mathbb{R}^k} \|r_0 - Q_{k+1} \tilde{H}_k y\|_2 \quad (\text{note that } \|\cdot\|_2 \text{ is unitary invariant}) \\
&= x_0 + Q_k \cdot \arg \min_{y \in \mathbb{R}^k} \|Q_{k+1}^H r_0 - \tilde{H}_k y\|_2 \quad (\text{note that } q_1 = \frac{r_0}{\|r_0\|_2}, \text{ while } q_2, \dots, q_{k+1} \text{ are orthogonal to } r_0) \\
&= x_0 + Q_k \cdot \arg \min_{y \in \mathbb{R}^k} \|\|r_0\|_2 e_1 - \tilde{H}_k y\|_2
\end{aligned}$$

其中 e_1 代表 $\mathbb{R}^{k+1}$ 的第一个标准基向量.

因此 GMRES 算法每一步的近似解都等价于求解一个最小二乘问题:

$$y_k := \arg \min_{y \in \mathbb{R}^k} \| \|r_0\|_2 e_1 - \tilde{H}_k y \|_2$$

$$x_k := x_0 + Q_k y_k$$

4.2.2 子问题

GMRES 算法每一步的近似解都等价于求解一个最小二乘问题:

$$y_k := \arg \min_{y \in \mathbb{R}^k} \| \|r_0\|_2 e_1 - \tilde{H}_k y \|_2$$

$$x_k := x_0 + Q_k y_k$$

其中 e_1 代表 $\mathbb{R}^{k+1}$ 的第一个标准基向量.

注意到 $\tilde{H}_k \in \mathbb{R}^{(k+1) \times k}$ 是一个上 Hessenberg 矩阵,
因此这个最小二乘问题可以使用 Givens QR 分解来求解.

以 $k = 4$ 的情况为例:

$$\begin{aligned} \tilde{H}_k y &= \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ & * & * & * \\ & & * & * \\ & & & * \end{bmatrix} \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \end{bmatrix} = \begin{bmatrix} \|r_0\|_2 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \|r_0\|_2 e_1 \\ G_{1,2} \tilde{H}_k y &= \begin{bmatrix} * & * & * & * \\ \boxed{0} & * & * & * \\ & * & * & * \\ & & * & * \\ & & & * \end{bmatrix} \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \end{bmatrix} = \begin{bmatrix} * \\ \boxed{*} \\ 0 \\ 0 \\ 0 \end{bmatrix} = G_{1,2} \|r_0\|_2 e_1 \\ G_{2,3} G_{1,2} \tilde{H}_k y &= \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ & \boxed{0} & * & * \\ & & * & * \\ & & & * \end{bmatrix} \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \end{bmatrix} = \begin{bmatrix} * \\ * \\ \boxed{*} \\ 0 \\ 0 \end{bmatrix} = G_{2,3} G_{1,2} \|r_0\|_2 e_1 \\ G_{3,4} G_{2,3} G_{1,2} \tilde{H}_k y &= \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ & 0 & * & * \\ & & \boxed{0} & * \\ & & & * \end{bmatrix} \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \end{bmatrix} = \begin{bmatrix} * \\ * \\ * \\ \boxed{*} \\ 0 \end{bmatrix} = G_{3,4} G_{2,3} G_{1,2} \|r_0\|_2 e_1 \\ G_{4,5} G_{3,4} G_{2,3} G_{1,2} \tilde{H}_k y &= \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ & 0 & * & * \\ & & 0 & * \\ & & & \boxed{0} \end{bmatrix} \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \end{bmatrix} = \begin{bmatrix} * \\ * \\ * \\ * \\ \boxed{*} \end{bmatrix} = G_{4,5} G_{3,4} G_{2,3} G_{1,2} \|r_0\|_2 e_1 \end{aligned}$$

在上述示例中我们很明显看到残差 $\|r_0\|_2$ 在 Givens 变换的作用下不断向下分裂.

由于 Givens 变换是酉变换, 故残差向量整体的 l_2 范数在 Givens 变换下是不变的.

因此每轮迭代向下分裂的残差一定不会超过上一轮迭代向下分裂的残差.

此外, 通过 k 次 Givens 变换分裂到最后一个 (即第 $k+1$ 个) 分量的残差我们是管不了的.

我们能管的只有变换后的残差向量的前 k 个分量.

换言之, 最小二乘解即为 k 阶上三角方程组的解:

$$\text{Least Squares: } G_{4,5}G_{3,4}G_{2,3}G_{1,2}\tilde{H}_ky = \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ & 0 & * & * \\ & & 0 & * \\ & & & \boxed{0} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} * \\ * \\ * \\ * \\ \boxed{*} \end{bmatrix} = G_{4,5}G_{3,4}G_{2,3}G_{1,2}\|r_0\|_2e_1$$

$$\Rightarrow$$

$$\text{Solve } \begin{bmatrix} * & * & * & * \\ & * & * & * \\ & & * & * \\ & & & * \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} * \\ * \\ * \\ * \end{bmatrix}$$

通过回代法即可得到最小二乘解 $y_k \in \mathbb{R}^k$

进而得到第 k 轮迭代的近似解 $x_k = x_0 + Q_k y_k$

(更严格的叙述)

记 $\tilde{H}_k \in \mathbb{R}^{(k+1) \times k}$ 通过 Givens 变换得到的 QR 分解为:

$$\tilde{H}_k = G_k^T \begin{bmatrix} R_k \\ 0_k^T \end{bmatrix}$$

其中 $G_k \in \mathbb{R}^{(k+1) \times (k+1)}$ 是累积的 k 个 Givens 变换, 而 $R_k \in \mathbb{R}^{k \times k}$ 是上三角阵.

将经过 Givens 变换的残差向量 $G_k\|r_0\|_2e_1$ 划分为:

$$G_k\|r_0\|_2e_1 = \begin{bmatrix} \eta_k \\ \varepsilon_k \end{bmatrix} \text{ where } \begin{cases} \eta_k \in \mathbb{R}^k \\ \varepsilon_k \in \mathbb{R} \end{cases}$$

其中 e_1 代表 $\mathbb{R}^{k+1}$ 的第一个标准基向量.

于是我们有:

$$\begin{aligned} y_k &:= \arg \min_{y \in \mathbb{R}^k} \|\|r_0\|_2e_1 - \tilde{H}_ky\|_2 \\ &= \arg \min_{y \in \mathbb{R}^k} \|G_k\|r_0\|_2e_1 - G_k\tilde{H}_ky\|_2 \\ &= \arg \min_{y \in \mathbb{R}^k} \left\| \begin{bmatrix} \eta_k \\ \varepsilon_k \end{bmatrix} - \begin{bmatrix} R_k \\ 0_k^T \end{bmatrix} y \right\|_2 \\ &= \arg \min_{y \in \mathbb{R}^k} \left\| \begin{bmatrix} \eta_k - R_k y \\ \varepsilon_k \end{bmatrix} \right\|_2 \\ &= \text{solution of } \{R_k y = \eta_k\} \end{aligned}$$

通过回代法即可得到最小二乘解 $y_k \in \mathbb{R}^k$

进而得到第 k 轮迭代的近似解 $x_k = x_0 + Q_k y_k$

记第 k 轮迭代的残差向量为 $r_k := b - Ax_k$

可以证明 $\|r_k\|_2 = |\varepsilon_k|$:

$$\begin{aligned} \|r_k\|_2 &= \|b - Ax_k\|_2 \\ &= \|b - A(x_0 + Q_k y_k)\|_2 \\ &= \|r_0 - AQ_k y_k\|_2 \\ &= \|r_0 - Q_{k+1} \tilde{H}_k y_k\|_2 \quad (\text{note that } \|\cdot\|_2 \text{ is unitary invariant}) \\ &= \|Q_{k+1}^H r_0 - \tilde{H}_k y_k\|_2 \quad (\text{note that } q_1 = \frac{r_0}{\|r_0\|_2}, \text{ while } q_2, \dots, q_{k+1} \text{ are orthogonal to } r_0) \\ &= \|\|r_0\|_2e_1 - \tilde{H}_k y_k\|_2 \\ &= \|G_k\|r_0\|_2e_1 - G_k\tilde{H}_k y_k\|_2 \\ &= \left\| \begin{bmatrix} \eta_k \\ \varepsilon_k \end{bmatrix} - \begin{bmatrix} R_k \\ 0_k^T \end{bmatrix} y_k \right\|_2 \\ &= \left\| \begin{bmatrix} \eta_k - R_k y_k \\ \varepsilon_k \end{bmatrix} \right\|_2 \quad (\text{note that } y_k = R_k^{-1} \eta_k) \\ &= \left\| \begin{bmatrix} 0_k \\ \varepsilon_k \end{bmatrix} \right\|_2 \\ &= |\varepsilon_k| \end{aligned}$$

因此迭代过程中我们无需显式地计算残差 r_k ，
可以直接将 $|\varepsilon_k|$ 作为 $\|r_k\|_2$ 的替代来进行收敛条件的判断。

4.2.3 收敛性

理论上，精确解 $x_* = A^{-1}b$ 一定位于 n 阶 Krylov 子空间中：

$$x_* = A^{-1}b \in \mathcal{K}(A, b, n) = \{x_0 + \mathcal{K}(A, r_0, n)\}$$

$$\text{where } r_0 := b - Ax_0$$

因此理论上 GMRES 在第 n 步迭代一定收敛 (换言之，理论上 GMRES 是直接法)
但实际应用时由于舍入误差和正交性损失等因素的影响，
我们没有机会走到底，因此只能将其作为迭代法来使用。

此外，理论上我们只能保证 GMRES 在第 n 步收敛，
至于前 $n - 1$ 步的收敛性我们是不知道的 (即不由系数矩阵的特征值或奇异值刻画)
考虑下面的例子：

$$\begin{bmatrix} 0 & & 1 \\ 1 & 0 & \\ & 1 & 0 \\ & & 1 & 0 \end{bmatrix} x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

其精确解为 $x_* = [0, 0, 0, 1]^T$
若选择零向量作为初始向量 x_0 ，则我们有：

$$r_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad Ar_0 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad A^2 r_0 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad A^3 r_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

因此 $x_* \perp \mathcal{K}(A, r_0, 3)$ 而 $x_* \in \mathcal{K}(A, r_0, 4)$
说明 GMRES 前 3 步的残差不变 (即没有改进)，而到第 4 步收敛 (即残差为零)，
而系数矩阵是一个酉矩阵，条件数已经好到家了，依然出现了上述糟糕的收敛表现。

(Lucky Breakdown)

注意到当且仅当 Arnoldi 过程提前中止时，GMRES 算法会出现提前中止。

设第 k 步 Arnoldi 过程提前中止了，

即 $Aq_k = A^k q_1 = A^k \frac{r_0}{\|r_0\|_2}$ 可以表示为 $q_1, q_2, \dots, q_k$ 的线性组合 (即 $Aq_k \in \mathcal{K}(A, r_0, k)$)

因此 q_{k+1} 生成不出来，同时 $H_{k+1,k} = 0$

于是我们有：

$$\tilde{H}_k = \begin{bmatrix} H_k \\ 0_k^T \end{bmatrix}$$

此时子问题 (一个最小二乘问题) 就等价于一个线性方程组的求解问题：

$$y_k := \arg \min_{y \in \mathbb{R}^k} \|\|r_0\|_2 e_1 - \tilde{H}_k y\|_2$$

$$= \text{solution of } \{H_k y = \|r_0\|_2 e_1\}$$

注意第二个式子中的 $e_1 \in \mathbb{R}^k$ 要比第一个式子中的 $e_1 \in \mathbb{R}^{k+1}$ 少一维。

由于 $H_k \in \mathbb{R}^{k \times k}$ 仍是上 Hessenberg 矩阵，

因此我们依然按照 Givens QR 的方法求解子问题。

记 $H_k y = \|r_0\|_2 e_1$ 的解为 $y_k \in \mathbb{R}^{k \times k}$

值得注意的是，从上一节的观点来看，

由于 $H_{k+1,k} = 0$ ，因此我们不需要第 k 个 Givens 变换 (或者说它可以取成单位阵的形式)

这说明残差向量 $\|r_0\|_2 e_1$ 只接受 $k - 1$ 次 Givens 变换的分裂，

所以其第 $k + 1$ 个分量 (也就是我们之前管不了的那个分量，记号为 ε_k) 为零。

理论上，此时仍然有 $\|r_k\|_2 = |\varepsilon_k| = 0$ 成立。

这说明第 k 步的近似解 $x_k = x_0 + Q_k y_k$ 就是 $Ax = b$ 的精确解。

因此我们可以提前中止 GMRES 算法.
能发生这样的提前中止是幸运的, 我们称之为 lucky breakdown.

4.2.4 实用形式

注意到无论是否发生 lucky breakdown,
我们都需要通过 Givens 变换求解子问题, 即涉及 $\tilde{H}_k$ 的 Givens QR 分解.
注意到第 k 步迭代的子问题中, 前 $k - 1$ 个 Givens 变换与上一轮迭代相同.
因此我们不妨将这些 Givens 变换记录下来, 每步求解子问题只需计算一个 Givens 变换即可.
这能使得求解一次子问题的计算复杂度从 $O(k^2)$ 降到 $O(k)$
即使加上一次使用回代法计算 y_k 的计算复杂度 $O(k^2)$,
在 k 步 GMRES 迭代中求解子问题的计算复杂度依然是 $O(k^2)$ 级别.

因此 k 步 GMRES 迭代的计算复杂度为 $O(nk^2)$ (主要是 Arnoldi 过程的代价)
存储复杂度为 $O(nk)$ (主要是 $Q_k \in \mathbb{R}^{n \times k}$ 的存储)
这说明通过记录并复用 Givens 变换, 我们成功让求解子问题的代价被主问题的代价 "吃" 掉了.

```

function:  $x = \text{GMRES}(A, b, \text{max\_iter}, \text{tolerance})$ 
 $n = \dim(A)$ 
 $Q = \text{zeros}(n, \text{max\_iter} + 1)$  (orthonormal basis vectors)
 $Q(:, 1) = \frac{b}{\|b\|_2}$ 
 $H = \text{zeros}(\text{max\_iter} + 1, \text{max\_iter})$  (Hessenberg matrix)
 $\delta = \text{zeros}(n, 1)$  (temporary storage for orthogonalization)
reorthogonalization_loop = 2
 $G = \text{zeros}(\text{max\_iter}, 2)$  (Givens rotation coefficients storage)
 $r = \text{zeros}(\text{max\_iter} + 1, 1)$  (residuals in Givens-rotated space)
 $r(1) = \|b\|_2$ 


---


for  $k = 1 : \text{max\_iter}$ 
     $Q(:, k+1) = A \cdot Q(:, k)$  (expand Krylov subspace)
    for iter = 1 : reorthogonalization_loop
        for  $i = 1 : k$  (Modified Gram-Schmidt orthogonalization)
             $\delta(i) = Q(:, i)^T Q(:, k+1)$ 
             $H(i, k) = H(i, k) + \delta(i)$ 
             $Q(:, k+1) = Q(:, k+1) - \delta(i)Q(:, i)$ 
        end
    end
    end
    for  $j = 1 : k - 1$  (apply Givens rotations to new column)
         $[c, s] = G(j, 1 : 2)$ 
         $H(j : j+1, k) = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} H(j : j+1, k)$ 
    end
     $H(k+1, k) = \|Q(:, k+1)\|_2$  (compute norm of  $Q(:, k+1)$ )
    if  $H(k+1, k) < 10^{-10}$  (lucky breakdown)
        break
    else
         $Q(:, k+1) = Q(:, k+1) / H(k+1, k)$  (normalize basis vector)
         $[c, s] = \text{Givens}(H(k, k), H(k+1, k))$  (compute Givens rotation coefficients)
         $G(k, 1 : 2) = [c, s]$ 
         $H(k : k+1, k) = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} H(k : k+1, k)$ 
         $r(k : k+1) = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} r(k : k+1)$ 
        if  $|r(k+1)| < \text{tolerance}$  (check for convergence)
            break
        end
    end
end


---


 $y = \text{Backward\_Sweep}(H(1 : k, 1 : k), r(1 : k))$  (solve reduced system)
 $x = Q(:, 1 : k)y$  (compute final solution)
end

```

Matlab 实现参见 Homework 12 Problem 1

4.2.5 对比: FOM

全正交法 (Fully Orthogonal Method, FOM) 是处理非对称矩阵的残量正交化方法 (现已淘汰).

我们之所以介绍它, 是因为它和**广义极小化残量法** (GMRES) 非常相像.

下面我们对比二者在 k 阶 Krylov 子空间 $\mathcal{K}(A, b, k)$ 寻找 $x_* = A^{-1}b$ 的 "最佳近似" 的准则:

- ① GMRES (残量极小化)

$$x_k := \arg \min_{x \in x_0 + \mathcal{K}(A, r_0, k)} \|b - Ax\|_2$$

- ② FOM (残量正交化)

$$\begin{aligned}
x_k &= x \in x_0 + K(A, r_0, k) \text{ such that } (b - Ax) \perp \mathcal{K}(A, r_0, k) \\
&= x_0 + \text{projection of } r_0 \in \mathbb{C}^n \text{ on } \mathcal{K}(A, r_0, k) \\
&= x \in x_0 + K(A, r_0, k) \text{ such that } Q_k^H(b - Ax) = 0_k
\end{aligned}$$

给定 $A \in \mathbb{R}^{n \times n}$ 和 $b \in \mathbb{R}^n$

设初始向量为 $x_0 \in \mathbb{R}^n$, 记初始残差向量为 $r_0 := b - Ax_0$

假设已经通过 Arnoldi 过程得到了 k 阶 Krylov 子空间 $\mathcal{K}(A, r^{(0)}, x)$ 的一组标准正交基 $Q_k = [q_1, \dots, q_k]$, 满足:

$$\begin{aligned}
AQ_k &= A[q_1, \dots, q_k] \\
&= [q_1, \dots, q_k, q_{k+1}] \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1k} \\ h_{21} & h_{22} & \cdots & h_{2k} \\ & \ddots & \ddots & \vdots \\ & & h_{k,k-1} & h_{k,k} \\ & & & h_{k+1,k} \end{bmatrix} \\
&= Q_{k+1} \tilde{H}_k \\
&= [q_1, \dots, q_k] \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1k} \\ h_{21} & h_{22} & \cdots & h_{2k} \\ & \ddots & \ddots & \vdots \\ & & h_{k,k-1} & h_{k,k} \end{bmatrix} + q_{k+1} h_{k+1,k} \\
&= Q_k H_k + \text{rank-one} \quad (\text{where rank-one} := q_{k+1} h_{k+1,k})
\end{aligned}$$

我们的目标是将残量正交化.

设 $x_k = x_0 + Q_k y_k$ (其中 $y_k \in \mathbb{R}^k$)

于是我们有:

$$\begin{aligned}
0_k &= Q_k^T(b - Ax_k) \\
&= Q_k^T(b - A(x_0 + Q_k y_k)) \\
&= Q_k^T(r_0 - AQ_k y_k) \\
&= Q_k^T r_0 - Q_k^T A Q_k y_k \quad (\text{note that } q_1 = \frac{r_0}{\|r_0\|_2}, \text{ while } q_2, \dots, q_{k+1} \text{ are orthogonal to } r_0) \\
&= \|r_0\|_2 e_1 - H_k y_k
\end{aligned}$$

其中 e_1 是 $\mathbb{R}^k$ 的第一个标准正交基向量.

因此 $y_k \in \mathbb{R}^k$ 就是线性方程组 $H_k y = \|r_0\|_2 e_1$ 的精确解.

注意到 $H_k \in \mathbb{R}^{k \times k}$ 是上 Hessenberg 矩阵, 故可以使用 Givens QR 来求解 (实际上也可用 LU 分解来解)

以 $k = 4$ 的情况为例:

$$\begin{aligned}
H_k y &= \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ & * & * & * \\ & & * & * \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} \|r_0\|_2 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \|r_0\|_2 e_1 \\
G_{1,2} \tilde{H}_k y &= \begin{bmatrix} * & * & * & * \\ \boxed{0} & * & * & * \\ & * & * & * \\ & & * & * \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} * \\ \boxed{*} \\ 0 \\ 0 \end{bmatrix} = G_{1,2} \|r_0\|_2 e_1 \\
G_{2,3} G_{1,2} \tilde{H}_k y &= \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ & \boxed{0} & * & * \\ & & * & * \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} * \\ * \\ \boxed{*} \\ 0 \end{bmatrix} = G_{2,3} G_{1,2} \|r_0\|_2 e_1 \\
G_{3,4} G_{2,3} G_{1,2} \tilde{H}_k y &= \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ & 0 & * & * \\ & & \boxed{0} & * \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} * \\ * \\ * \\ \boxed{*} \end{bmatrix} = G_{3,4} G_{2,3} G_{1,2} \|r_0\|_2 e_1
\end{aligned}$$

通过回代法即可得到最小二乘解 $y_k \in \mathbb{R}^k$

进而得到第 k 轮迭代的近似解 $x_k = x_0 + Q_k y_k$

对应的残差范数为:

$$\begin{aligned}
\|r_k\|_2 &= \|b - Ax_k\|_2 \\
&= \|b - A(x_0 + Q_k y_k)\|_2 \\
&= \|r_0 - AQ_k y_k\|_2 \\
&= \|r_0 - Q_{k+1} \tilde{H}_k y_k\|_2 \\
&= \|Q_{k+1}^T r_0 - \tilde{H}_k y_k\|_2 \quad (\text{note that } q_1 = \frac{r_0}{\|r_0\|_2}, \text{ while } q_2, \dots, q_{k+1} \text{ are orthogonal to } r_0) \\
&= \|\|r_0\|_2 e_1 - \tilde{H}_k y_k\|_2 \quad (\text{where } e_1 \in \mathbb{R}^{k+1}) \\
&= \left\| \begin{bmatrix} \|r_0\|_2 e_1 - H_k y_k \\ 0 - h_{k+1,k} y_k^{(k)} \end{bmatrix} \right\|_2 \quad (\text{where } e_1 \in \mathbb{R}^k, \text{ while } y_k^{(k)} \text{ is the } k\text{-th element of } y_k := H_k^{-1} \|r_0\|_2 e_1) \\
&= \left\| \begin{bmatrix} 0_k \\ h_{k+1,k} y_k^{(k)} \end{bmatrix} \right\|_2 \\
&= |h_{k+1,k} y_k^{(k)}|
\end{aligned}$$

记 $H_k \in \mathbb{R}^{k \times k}$ 通过 Givens 变换得到的 QR 分解为:

$$H_k = G_k^T R_k$$

其中 $G_k \in \mathbb{R}^{k \times k}$ 是累积的 $k - 1$ 个 Givens 变换, 而 $R_k \in \mathbb{R}^{k \times k}$ 是上三角阵. 我们记 $\eta_k = G_k \cdot \|r_0\|_2 e_1$, 则我们有:

$$\begin{aligned}
y_k &= H_k^{-1} \|r_0\|_2 e_1 \\
&= H_k^{-1} G_k^T G_k \|r_0\|_2 e_1 \\
&= (G_k H_k)^{-1} (G_k \|r_0\|_2 e_1) \\
&= R_k^{-1} \cdot \eta_k
\end{aligned}$$

根据回代法的求解思路, 我们知道 y_k 的第 k 个元素 $y_k^{(k)}$ 即为:

$$y_k^{(k)} = \frac{\eta_k^{(k)}}{(R_k)_{[k,k]}}$$

其中 $\eta_k^{(k)}$ 是 η_k 的第 k 个元素.

因此第 k 步 FOM 迭代的残差范数为:

$$\begin{aligned}
\|r_k\|_2 &= |h_{k+1,k} y_k^{(k)}| \\
&= \left| h_{k+1,k} \cdot \frac{\eta_k^{(k)}}{(R_k)_{[k,k]}} \right|
\end{aligned}$$

与之对比, 在 GMRES 中 $y_k \in \mathbb{R}^k$ 是线性方程组 $\tilde{H}_k y = \|r_0\|_2 e_1$ 的最小二乘解

(注意: 这里的 e_1 是 $\mathbb{R}^{k+1}$ 的第一个标准正交基向量)

可以发现: FOM 和 GMRES 的唯一区别就在于 $\tilde{H}_k$ 有没有进行第 k 和 $k + 1$ 行的 Givens 变换.

因此我们只要稍微修改 GMRES 算法就能得到 FOM 算法.

具体来说, 我们将修改检验残差范数的停止条件, 并将 Givens 变换移到 else 语句中即可:

(存疑: 修改得是否正确?)

```

function:  $x = \text{FOM}(A, b, \text{max\_iter}, \text{tolerance})$ 
 $n = \text{dim}(A)$ 
 $Q = \text{zeros}(n, \text{max\_iter} + 1)$  (orthonormal basis vectors)
 $Q(:, 1) = \frac{b}{\|b\|_2}$ 
 $H = \text{zeros}(\text{max\_iter} + 1, \text{max\_iter})$  (Hessenberg matrix)
 $\delta = \text{zeros}(n, 1)$  (temporary storage for orthogonalization)
reorthogonalization_loop = 2
 $G = \text{zeros}(\text{max\_iter}, 2)$  (Givens rotation coefficients storage)
 $r = \text{zeros}(\text{max\_iter} + 1, 1)$  (residuals in Givens-rotated space)
 $r(1) = \|b\|_2$ 


---


for  $k = 1 : \text{max\_iter}$ 
     $Q(:, k+1) = A \cdot Q(:, k)$  (expand Krylov subspace)
    for iter = 1 : reorthogonalization_loop
        for  $i = 1 : k$  (Modified Gram-Schmidt orthogonalization)
             $\delta(i) = Q(:, i)^T Q(:, k+1)$ 
             $H(i, k) = H(i, k) + \delta(i)$ 
             $Q(:, k+1) = Q(:, k+1) - \delta(i)Q(:, i)$ 
        end
    end
    for  $j = 1 : k - 1$  (apply Givens rotations to new column)
         $[c, s] = G(j, 1 : 2)$ 
         $H(j : j+1, k) = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} H(j : j+1, k)$ 
    end
     $H(k+1, k) = \|Q(:, k+1)\|_2$  (compute norm of  $Q(:, k+1)$ )
    if  $H(k+1, k) < 10^{-10}$  (lucky breakdown)
        break
    else
         $Q(:, k+1) = Q(:, k+1) / H(k+1, k)$  (normalize basis vector)
        if  $\left| H(k+1, k) \frac{r(k)}{H(k, k)} \right| < \text{tolerance}$  (check for convergence)
            break
        else
             $[c, s] = \text{Givens}(H(k, k), H(k+1, k))$  (compute Givens rotation coefficients)
             $G(k, 1 : 2) = [c, s]$ 
             $H(k : k+1, k) = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} H(k : k+1, k)$ 
             $r(k : k+1) = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} r(k : k+1)$ 
        end
    end
end


---


 $y = \text{Backward\_Sweep}(H(1 : k, 1 : k), r(1 : k))$  (solve reduced system)
 $x = Q(:, 1 : k)y$  (compute final solution)
end

```

4.3 共轭梯度法

使用超松弛迭代法求解线性方程组 $Ax = b$ 时，我们需要确定松弛因子 ω 。

但只有系数矩阵 A 具有较好的性质 (例如 A 对称正定且具有相容次序) 时，我们才有可能找到最佳松弛因子 ω_{opt} 。更何况计算 ω_{opt} 时需要首先求出 Jacobi 迭代矩阵的谱半径 $\rho(B^{(1)})$ (这通常是非常困难的)。

我们将介绍一种不需要确定任何参数的求解对称正定线性方程组的方法——共轭梯度法。它已经成为求解大型稀疏线性方程组最受欢迎的一类方法。

共轭梯度法有多种引入方法，这里我们采用较为直观的最优化问题来引入。为此，我们首先介绍最速下降法。

4.3.1 最速下降法

考虑线性方程组 $Ax = b$ 的求解问题.

其中 $A \in \mathbb{R}^{n \times n}$ 是给定的对称正定阵.

定义二次函数 $\varphi(x) = x^T Ax - 2b^T x$

(数值线性代数, 定理 5.1.1)

对称正定线性方程组 $Ax = b$ 的解等价于二次函数 $\varphi(x) = x^T Ax - 2b^T x$ 的极小值点 (它是唯一的, 因而是最小值点).

求解二次函数 $\varphi(x) = x^T Ax - 2b^T x$ 的极小值问题,

通常从一个初始向量 $x^{(0)}$ 出发, 按迭代格式 $x^{(k+1)} = x^{(k)} + t_k d^{(k)}$ 得到向量序列 $\{x^{(k)}\}$

不同的确定搜索方向 $d^{(k)}$ 和步长 t_k 的方法, 就得到不同的迭代算法.

考虑最速下降法:

- 固定下降方向 $d^{(k)}$, 考虑确定步长 t_k :

$$\begin{aligned} t_k &= \arg \min_{t \geq 0} \varphi(x^{(k)} + td^{(k)}) \\ &= \arg \left\{ \frac{d}{dt} \varphi(x^{(k)} + td^{(k)}) = 0 \right\} \quad (\text{note that } \varphi(x^{(k)} + td^{(k)}) \text{ is convex with respect to } t) \\ &= \arg \{ (d^{(k)})^T \nabla \varphi(x^{(k)} + td^{(k)}) = 0 \} \\ &= \arg \{ (d^{(k)})^T [2A(x^{(k)} + td^{(k)}) - 2b] = 0 \} \quad (\text{note that } \nabla \varphi(x) = 2Ax - 2b) \\ &= \arg \{ 2t(d^{(k)})^T Ad^{(k)} + 2(d^{(k)})^T (Ax^{(k)} - b) = 0 \} \\ &= \arg \{ 2t(d^{(k)})^T Ad^{(k)} - 2(d^{(k)})^T r^{(k)} = 0 \} \quad (\text{denote residual vector } r^{(k)} = b - Ax^{(k)}) \\ &= \frac{(d^{(k)})^T r^{(k)}}{(d^{(k)})^T Ad^{(k)}} \end{aligned}$$

因此步长 $t_k = \frac{(r^{(k)})^T d^{(k)}}{(d^{(k)})^T Ad^{(k)}}$ (其中残差向量 $r^{(k)} = b - Ax^{(k)}$)

那么 $\varphi(x^{(k+1)}) = \varphi(x^{(k)} + t_k d^{(k)})$ 在什么条件下小于 $\varphi(x^{(k)})$ 呢?

$$\begin{aligned} \varphi(x^{(k+1)}) - \varphi(x^{(k)}) &= \varphi(x^{(k)} + t_k d^{(k)}) - \varphi(x^{(k)}) \\ &= (x^{(k)} + t_k d^{(k)})^T A(x^{(k)} + t_k d^{(k)}) - 2b^T (x^{(k)} + t_k d^{(k)}) - [(x^{(k)})^T A x^{(k)} - 2b^T x^{(k)}] \\ &= t_k^2 (d^{(k)})^T A d^{(k)} + 2t_k (d^{(k)})^T (Ax^{(k)} - b) \\ &= t_k^2 (d^{(k)})^T A d^{(k)} - 2t_k (d^{(k)})^T r^{(k)} \quad (\text{denote residual vector } r^{(k)} = b - Ax^{(k)}) \\ &= \left(\frac{(d^{(k)})^T r^{(k)}}{(d^{(k)})^T Ad^{(k)}} \right)^2 (d^{(k)})^T A d^{(k)} - 2 \frac{(d^{(k)})^T r^{(k)}}{(d^{(k)})^T Ad^{(k)}} (d^{(k)})^T r^{(k)} \\ &= - \frac{[(d^{(k)})^T r^{(k)}]^2}{(d^{(k)})^T A d^{(k)}} \\ &\leq 0 \end{aligned}$$

上式当且仅当 $(r^{(k)})^T d^{(k)} \neq 0$ 时严格成立.

因此只要 $(r^{(k)})^T d^{(k)} \neq 0$, 就有 $\varphi(x^{(k+1)}) < \varphi(x^{(k)})$ 成立.

- 再考虑确定下降方向 $d^{(k)}$:

根据 $\varphi(x)$ 在 $x^{(k)}$ 的一阶 Taylor 展开式 $\varphi(x) = \varphi(x^{(k)}) + \nabla \varphi(x^{(k)})^T (x - x^{(k)}) + O(\|x - x^{(k)}\|)$ 可知,

在 $x^{(k)}$ 的足够小的邻域内, 位移 $x - x^{(k)}$ 沿负梯度方向 $-\nabla \varphi(x^{(k)})$ 时下降最快

因此我们可取 $d^{(k)} = -\nabla \varphi(x^{(k)}) = -(2Ax^{(k)} - 2b) = 2r^{(k)}$

(为与教材保持一致, 我们丢弃系数 2, 取 $d^{(k)} = r^{(k)}$)

根据之前的结论, 对应的步长 $t_k = \frac{(r^{(k)})^T d^{(k)}}{(d^{(k)})^T A d^{(k)}} = \frac{(r^{(k)})^T r^{(k)}}{(r^{(k)})^T A r^{(k)}}$

只要 $(r^{(k)})^T d^{(k)} = (r^{(k)})^T r^{(k)} = \|r^{(k)}\|_2^2 \neq 0$, 就有 $\varphi(x^{(k+1)}) < \varphi(x^{(k)})$ 成立.

综上所述, 我们得到如下算法:

(最速下降法, 数值线性代数, 算法 5.1.1)

Given positive definite matrix A , vector b and initial point $x^{(0)}$

$$r^{(0)} = b - Ax^{(0)}$$

for $k = 0 : \text{max_iter} - 1$

$$t_k = \frac{(r^{(k)})^T r^{(k)}}{(r^{(k)})^T A r^{(k)}}$$

$$x^{(k+1)} = x^{(k)} + t_k r^{(k)}$$

$$r^{(k+1)} = b - Ax^{(k+1)} = b - A(x^{(k)} + t_k r^{(k)}) = r^{(k)} - t_k (A r^{(k)}) \quad (\text{复用 } A r^{(k)} \text{ 以规避一次矩阵乘法})$$

$$k = k + 1$$

if $\|r^{(k)}\|_2 < \text{tolerance}$ (终止条件)

break

end

end

$$x = x^{(k)}$$

收敛性分析的准备工作:

(数值线性代数, 引理 5.1.1)

设 Hermite 正定阵 $A \in \mathbb{C}^{n \times n}$ 的特征值为 $0 < \lambda_1 \leq \dots \leq \lambda_n$

对于任意复系数多项式 $p(t)$ 我们都有:

$$\|p(A)x\|_A \leq \max_{1 \leq i \leq n} |p(\lambda_i)| \|x\|_A \quad (\forall x \in \mathbb{C}^n)$$

• **证明:**

设 A 的谱分解为 $A = U \Lambda U^H$

其中 $U \in \mathbb{C}^{n \times n}$ 为酉矩阵, $\Lambda := \text{diag}\{\lambda_1, \dots, \lambda_n\}$

对于任意 $x \in \mathbb{C}^n$, 我们都有:

$$\begin{aligned} \|p(A)x\|_A^2 &= \|A^{\frac{1}{2}} p(A)x\|_2^2 \\ &= \|U \Lambda^{\frac{1}{2}} U^H U p(\Lambda) U^H x\|_2^2 \quad (\text{note that } \|\cdot\|_2 \text{ is unitary invariant}) \\ &= \|\Lambda^{\frac{1}{2}} p(\Lambda) U^H x\|_2^2 \\ &= x^H U \overline{p(\Lambda)} \Lambda^{\frac{1}{2}} \Lambda^{\frac{1}{2}} p(\Lambda) U^H x \\ &= x^H U |p(\Lambda)|^2 \Lambda U^H x \\ &= \max_{1 \leq i \leq n} |p(\lambda_i)|^2 \cdot x^H U \Lambda U^H x \\ &= \max_{1 \leq i \leq n} |p(\lambda_i)|^2 \cdot x^H A x \\ &= \max_{1 \leq i \leq n} |p(\lambda_i)|^2 \|x\|_A^2 \end{aligned}$$

于是我们有:

$$\|p(A)x\|_A \leq \max_{1 \leq i \leq n} |p(\lambda_i)| \|x\|_A \quad (\forall x \in \mathbb{C}^n)$$

(Chebyshev 极小化引理)

任意给定 $b > a > 0$, 我们有:

$$\min_{\alpha \in \mathbb{R}} \max_{a \leq t \leq b} |1 - \alpha t| = \min_{\alpha > 0} \max_{a \leq t \leq b} |1 - \alpha t| = \frac{b-a}{b+a}$$

• **证明:**

为使 $\max_{a \leq t \leq b} |1 - \alpha t|$ 最小化, 我们令 $\alpha \geq 0$ 且端点值 $|1 - \alpha a| = |1 - \alpha b|$

◦ ① 若 $1 - \alpha a \geq 0$ 且 $1 - \alpha b \geq 0$, 则我们有 $1 - \alpha a = 1 - \alpha b$

解得 $\alpha = 0$, 对应的目标函数值为 1

◦ ② 若 $1 - \alpha a \geq 0$ 且 $1 - \alpha b \leq 0$, 则我们有 $1 - \alpha a = -(1 - \alpha b)$

解得 $\alpha = \frac{2}{a+b}$, 对应的目标函数值为 $\frac{b-a}{b+a}$

注意到 $\frac{b-a}{b+a} < 1$, 因此我们有:

$$\min_{\alpha \in \mathbb{R}} \max_{a \leq t \leq b} |1 - \alpha t| = \min_{\alpha > 0} \max_{a \leq t \leq b} |1 - \alpha t| = \frac{b-a}{b+a}$$

(梯度下降法的收敛性, 数值线性代数, 定理 5.1.2)

考虑求解对称正定线性方程组 $Ax = b$, 任意给定初始向量 $x^{(0)}$

设对称正定阵 $A \in \mathbb{R}^{n \times n}$ 的特征值为 $0 < \lambda_1 \leq \dots \leq \lambda_n$,

则由梯度下降法 (不仅仅是最速下降法) 产生的序列 $\{x^{(k)}\}$ 满足:

$$\begin{aligned}\|x^{(k)} - x^*\|_A &\leq \left(\frac{\lambda_n - \lambda_1}{\lambda_n + \lambda_1} \right)^k \|x^{(0)} - x^*\|_A \\ &= \left(\frac{\kappa_2(A) - 1}{\kappa_2(A) + 1} \right)^k \|x^{(0)} - x^*\|_A\end{aligned}$$

其中精确解 $x_* = A^{-1}b$, 而范数 $\|\cdot\|_A$ 的定义为 $\|x\|_A := \sqrt{x^T A x}$

条件数 $\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} = \frac{\lambda_n}{\lambda_1}$

特殊地, 若 A 是单位阵 (唯一的对称正定的酉矩阵), 则算法一步收敛.

- 上述定理表明:

从任意初始向量 $x^{(0)}$ 出发, 由最速下降法产生的序列 $\{x^{(k)}\}$ 总是收敛到对称正定线性方程组 $Ax = b$ 的精确解 $x = A^{-1}b$

其收敛速度的快慢由 $\frac{\lambda_n - \lambda_1}{\lambda_n + \lambda_1}$ 决定.

- 虽然最速下降法简单易用, 且能充分利用 A 的稀疏性, 但是当问题相当病态 (即 $\lambda_1 \ll \lambda_n \Rightarrow \frac{\lambda_n - \lambda_1}{\lambda_n + \lambda_1} \rightarrow 1$) 时, 其收敛速度会变得非常慢, 因此很少用于对称正定线性方程组 $Ax = b$ 的实际求解.

此外, 它是一种贪心的算法.

可以证明其下降方向 (即负梯度方向, 也即残差方向) 相互正交 (即 $(r^{(k+1)})^T r^{(k)} = 0$)

因此会呈现出锯齿状 (zig-zag) 收敛路径.

这表明最速下降法过分追求眼前利益 (局部的梯度信息), 缺少了全局的考量, 因而其收敛效率可能并不高.

然而它揭示了一种重要的思想, 开辟了一条全新的求解线性方程组的途径.

我们对最速下降法稍加改进, 就能得到著名的共轭梯度法.

- 证明:

对于任意 $\alpha > 0$ 我们都有:

$$\begin{aligned}\|x^{(k)} - x_*\|_A^2 &= \|x^{(k-1)} + \alpha r^{(k-1)} - x_*\|_A^2 \quad (\text{note that } r^{(k-1)} = b - Ax^{(k-1)} = A(x_* - x^{(k-1)})) \\ &= \|(I - \alpha A)(x^{(k-1)} - x_*)\|_A^2 \quad (\text{use lemma}) \\ &\leq \max_{1 \leq i \leq n} |1 - \alpha \lambda_i|^2 \|x^{(k-1)} - x_*\|_A^2 \\ &\leq \max_{\lambda_1 \leq t \leq \lambda_n} |1 - \alpha t|^2 \|x^{(k-1)} - x_*\|_A^2\end{aligned}$$

因此我们有:

$$\begin{aligned}\|x^{(k)} - x_*\|_A &\leq \min_{\alpha > 0} \max_{\lambda_1 \leq t \leq \lambda_n} |1 - \alpha t| \cdot \|x^{(k-1)} - x_*\|_A \quad (\text{use lemma}) \\ &= \frac{\lambda_n - \lambda_1}{\lambda_n + \lambda_1} \|x^{(k-1)} - x_*\|_A\end{aligned}$$

不断递推可得:

$$\|x^{(k)} - x^*\|_A \leq \left(\frac{\lambda_n - \lambda_1}{\lambda_n + \lambda_1} \right)^k \|x^{(0)} - x^*\|_A$$

其中初始点 $x^{(0)}$ 是任意的.

4.3.2 共轭梯度法

对最速下降法做简单的分析就会发现,

负梯度方向尽管是局部的最佳下降方向, 但从全局来看并非最佳.

这就促使我们寻找全局意义上更好的下降方向, 但每步确定该下降方向的代价不要太大.

共轭梯度法就是根据这一思想设计的, 其具体计算过程如下:

给定初始向量 $x^{(0)}$, $k = 0$ 时和最速下降法一致:

$$\begin{aligned}
d^{(0)} &= r^{(0)} = b - Ax^{(0)} \\
t_0 &= \frac{(r^{(0)})^T d^{(0)}}{(d^{(0)})^T A d^{(0)}} = \frac{(r^{(0)})^T r^{(0)}}{(r^{(0)})^T A r^{(0)}} \\
x^{(1)} &= x^{(0)} + t_0 d^{(0)} = x^{(0)} + t_0 r^{(0)} \\
r^{(1)} &= b - Ax^{(1)} = b - A(x^{(0)} + t_0 r^{(0)}) = r^{(0)} - t_0 A r^{(0)}
\end{aligned}$$

对第 $k \geq 1$ 步, 下降方向不再取负梯度方向 $-\nabla \varphi(x^{(k)}) = -(2Ax^{(k)} - 2b) = 2r^{(k)}$ (一般丢弃系数 2), 而是在 $r^{(k)}$ 和 $d^{(k-1)}$ 所张成的二维平面 $S_k = \{x = x^{(k)} + \xi r^{(k)} + \eta d^{(k-1)} : \xi, \eta \in \mathbb{R}\}$ 内找到使函数 φ 下降最快的方向作为新的下降方向 $d^{(k)}$

将 φ 限制在二维平面 $S_k = \{x = x^{(k)} + \xi r^{(k)} + \eta d^{(k-1)} : \xi, \eta \in \mathbb{R}\}$ 得到的新函数为:

$$\begin{aligned}
g(\xi, \eta) &= \varphi(x^{(k)} + \xi r^{(k)} + \eta d^{(k-1)}) \\
&= (x^{(k)} + \xi r^{(k)} + \eta d^{(k-1)})^T A (x^{(k)} + \xi r^{(k)} + \eta d^{(k-1)}) - 2b^T (x^{(k)} + \xi r^{(k)} + \eta d^{(k-1)})
\end{aligned}$$

其偏导数为:

$$\begin{aligned}
\frac{\partial g(\xi, \eta)}{\partial \xi} &= (r^{(k)})^T \nabla \varphi(x^{(k)} + \xi r^{(k)} + \eta d^{(k-1)}) \\
&= (r^{(k)})^T [2A(x^{(k)} + \xi r^{(k)} + \eta d^{(k-1)}) - 2b] \quad (\text{note that } r^{(k)} = b - Ax^{(k)}) \\
&= (r^{(k)})^T [2\xi A r^{(k)} + 2\eta A d^{(k-1)} - 2r^{(k)}] \\
&= 2[\xi (r^{(k)})^T A r^{(k)} + \eta (r^{(k)})^T A d^{(k-1)} - (r^{(k)})^T r^{(k)}] \\
\hline
\frac{\partial g(\xi, \eta)}{\partial \eta} &= (d^{(k-1)})^T \nabla \varphi(x^{(k)} + \xi r^{(k)} + \eta d^{(k-1)}) \\
&= (d^{(k-1)})^T [2A(x^{(k)} + \xi r^{(k)} + \eta d^{(k-1)}) - 2b] \quad (\text{note that } r^{(k)} = b - Ax^{(k)}) \\
&= (d^{(k-1)})^T [2\xi A r^{(k)} + 2\eta A d^{(k-1)} - 2r^{(k)}] \quad (\text{note that } (r^{(k)})^T d^{(k-1)} = 0) \\
&= 2[\xi (d^{(k-1)})^T A r^{(k)} + \eta (d^{(k-1)})^T A d^{(k-1)} - (d^{(k-1)})^T r^{(k)}] \\
&= 2[\xi (d^{(k-1)})^T A r^{(k)} + \eta (d^{(k-1)})^T A d^{(k-1)}]
\end{aligned}$$

我们验证 $(r^{(k)})^T d^{(k-1)} = 0$:

$$\begin{aligned}
(r^{(k)})^T d^{(k-1)} &= (b - Ax^{(k)})^T d^{(k-1)} \\
&= [b - A(x^{(k-1)} + t_{k-1} d^{(k-1)})]^T d^{(k-1)} \\
&= (r^{(k-1)} - t_{k-1} A d^{(k-1)})^T d^{(k-1)} \quad (\text{the best stepsize is } t_{k-1} = \frac{(r^{(k-1)})^T d^{(k-1)}}{(d^{(k-1)})^T A d^{(k-1)}}) \\
&= (r^{(k-1)})^T d^{(k-1)} - \frac{(r^{(k-1)})^T d^{(k-1)}}{(d^{(k-1)})^T A d^{(k-1)}} (d^{(k-1)})^T A d^{(k-1)} \\
&= (r^{(k-1)})^T d^{(k-1)} - (r^{(k-1)})^T d^{(k-1)} \\
&= 0
\end{aligned}$$

由于 φ 是一个凸二次函数,

故 φ 在二维平面 $S_k = \{x = x^{(k)} + \xi r^{(k)} + \eta d^{(k-1)} : \xi, \eta \in \mathbb{R}\}$ 中具有唯一的最小值点 $\tilde{x}$.

$$\text{令 } \begin{cases} \frac{\partial g(\xi, \eta)}{\partial \xi} = 2[\xi (r^{(k)})^T A r^{(k)} + \eta (r^{(k)})^T A d^{(k-1)} - (r^{(k)})^T r^{(k)}] = 0 \\ \frac{\partial g(\xi, \eta)}{\partial \eta} = 2[\xi (d^{(k-1)})^T A r^{(k)} + \eta (d^{(k-1)})^T A d^{(k-1)}] = 0 \end{cases} \quad \text{可解得:}$$

$$\tilde{x} = x^{(k)} + \tilde{\xi} r^{(k)} + \tilde{\eta} d^{(k-1)}$$

$$\text{where } \begin{cases} \tilde{\xi} (r^{(k)})^T A r^{(k)} + \tilde{\eta} (r^{(k)})^T A d^{(k-1)} = (r^{(k)})^T r^{(k)} \\ \tilde{\xi} (d^{(k-1)})^T A r^{(k)} + \tilde{\eta} (d^{(k-1)})^T A d^{(k-1)} = 0 \end{cases} \Rightarrow \frac{\tilde{\eta}}{\tilde{\xi}} = -\frac{(d^{(k-1)})^T A r^{(k)}}{(d^{(k-1)})^T A d^{(k-1)}}$$

显然当 $r^{(k)} \neq 0_n$ 时, 我们有 $\tilde{\xi} \neq 0$, 因此我们可取 $d^{(k)}$ 为:

$$d^{(k)} = \frac{1}{\tilde{\xi}} (\tilde{x} - x^{(k)}) = \frac{1}{\tilde{\xi}} (\tilde{\xi} r^{(k)} + \tilde{\eta} d^{(k-1)}) = r^{(k)} + \frac{\tilde{\eta}}{\tilde{\xi}} d^{(k-1)} = r^{(k)} - \frac{(d^{(k-1)})^T A r^{(k)}}{(d^{(k-1)})^T A d^{(k-1)}} d^{(k-1)}$$

我们验证 $(d^{(k)})^T A d^{(k-1)} = 0$:

$$\begin{aligned}
(d^{(k)})^T A d^{(k-1)} &= \left[r^{(k)} - \frac{(d^{(k-1)})^T A r^{(k)}}{(d^{(k-1)})^T A d^{(k-1)}} d^{(k-1)} \right]^T A d^{(k-1)} \\
&= (r^{(k)})^T A d^{(k-1)} - \frac{(d^{(k-1)})^T A r^{(k)}}{(d^{(k-1)})^T A d^{(k-1)}} (d^{(k-1)})^T A d^{(k-1)} \\
&= (r^{(k)})^T A d^{(k-1)} - (d^{(k-1)})^T A r^{(k)} \\
&= 0
\end{aligned}$$

也就是说，相邻两次迭代的下降方向 $d^{(k)}$ 和 $d^{(k-1)}$ 是相互共轭的 (关于 A 的内积为 0)

这样我们就知道了 $d^{(k-1)}$, $r^{(k)}$ 和 $d^{(k)}$ 之间存在关系 $\begin{cases} (r^{(k)})^T d^{(k-1)} = 0 \\ (d^{(k)})^T A d^{(k-1)} = 0 \end{cases}$

其几何意义如同所示 (记号不一致, 待修改):

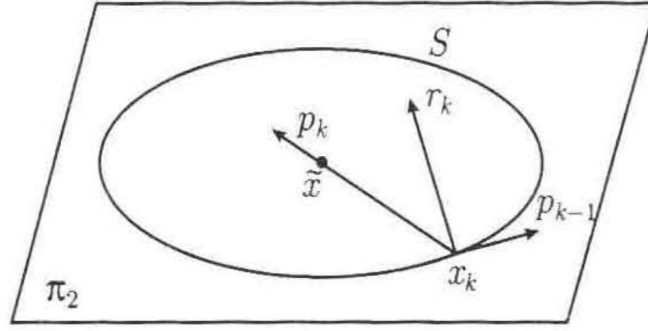


图 5.1

综上所述，我们得到如下的计算公式 ($k \geq 0$):

$$\begin{aligned}
d^{(0)} &= r^{(0)} = b - Ax^{(0)} \\
t_k &= \frac{(r^{(k)})^T d^{(k)}}{(d^{(k)})^T A d^{(k)}} \\
x^{(k+1)} &= x^{(k)} + t_k d^{(k)} \\
r^{(k+1)} &= b - Ax^{(k+1)} = r^{(k)} - t_k A d^{(k)} \\
\beta_k &= -\frac{(r^{(k+1)})^T A d^{(k)}}{(d^{(k)})^T A d^{(k)}} \\
d^{(k+1)} &= r^{(k+1)} + \beta_k d^{(k)}
\end{aligned}$$

在实际计算中，通常将上述公式进一步简化，从而得到形式上更简单且对称的计算公式。

- 简化 t_k 的计算公式:

$$\begin{aligned}
(r^{(k)})^T d^{(k)} &= (r^{(k)})^T (r^{(k)} + \beta_{k-1} d^{(k-1)}) \\
&= (r^{(k)})^T r^{(k)} + \beta_{k-1} (r^{(k)})^T d^{(k-1)} \quad (\text{note that } (r^{(k)})^T d^{(k-1)} = 0) \\
&= (r^{(k)})^T r^{(k)}
\end{aligned}$$

$$\text{因此 } t_k = \frac{(r^{(k)})^T d^{(k)}}{(d^{(k)})^T A d^{(k)}} = \frac{(r^{(k)})^T r^{(k)}}{(d^{(k)})^T A d^{(k)}}$$

- 简化 $r^{(k+1)}$ 的计算公式:

$$\begin{aligned}
r^{(k+1)} &= b - Ax^{(k+1)} \\
&= b - A(x^{(k)} + t_k d^{(k)}) \\
&= r^{(k)} - t_k A d^{(k)}
\end{aligned}$$

注意到 $A d^{(k)}$ 在计算 $t_k = \frac{(r^{(k)})^T d^{(k)}}{(d^{(k)})^T A d^{(k)}}$ 时已经计算过了。

- 简化 β_k 的计算公式:

由 $r^{(k+1)} = r^{(k)} - t_k A d^{(k)}$ 我们得到:

$$\begin{aligned}
(r^{(k+1)})^T Ad^{(k)} &= (r^{(k+1)})^T \cdot \frac{1}{t_k} (r^{(k)} - r^{(k+1)}) \\
&= \frac{1}{t_k} [(r^{(k+1)})^T r^{(k)} - (r^{(k+1)})^T r^{(k+1)}] \quad (\text{note that } (r^{(k+1)})^T r^{(k)} = 0) \\
&= -\frac{1}{t_k} (r^{(k+1)})^T r^{(k+1)} \quad (\text{note that } t_k = \frac{(r^{(k)})^T d^{(k)}}{(d^{(k)})^T Ad^{(k)}} = \frac{(r^{(k)})^T r^{(k)}}{(d^{(k)})^T Ad^{(k)}}) \\
&= -\frac{1}{\frac{(r^{(k)})^T r^{(k)}}{(d^{(k)})^T Ad^{(k)}}} (r^{(k+1)})^T r^{(k+1)} \\
&= -\frac{(r^{(k+1)})^T r^{(k+1)}}{(r^{(k)})^T r^{(k)}} (d^{(k)})^T Ad^{(k)}
\end{aligned}$$

$$\text{因此 } \beta_k = -\frac{(r^{(k+1)})^T Ad^{(k)}}{(d^{(k)})^T Ad^{(k)}} = -\frac{-\frac{(r^{(k+1)})^T r^{(k+1)}}{(r^{(k)})^T r^{(k)}} (d^{(k)})^T Ad^{(k)}}{(d^{(k)})^T Ad^{(k)}} = \frac{(r^{(k+1)})^T r^{(k+1)}}{(r^{(k)})^T r^{(k)}}$$

我们验证 $(r^{(k+1)})^T r^{(k)} = 0$:

$$\begin{aligned}
(r^{(k+1)})^T r^{(k)} &= (r^{(k+1)})^T (d^{(k)} - \beta_{k-1} d^{(k-1)}) \quad (\text{note that } d^{(k+1)} = r^{(k+1)} + \beta_k d^{(k)}) \\
&= (r^{(k+1)})^T d^{(k)} - \beta_{k-1} (r^{(k+1)})^T d^{(k-1)} \quad (\text{note that } (r^{(k+1)})^T d^{(k)} = (r^{(k+1)})^T d^{(k-1)} = 0) \\
&= 0
\end{aligned}$$

综上所述，简化后的计算公式为 ($k \geq 0$):

$$\begin{aligned}
&\overline{d^{(0)} = r^{(0)} = b - Ax^{(0)}} \\
&t_k = \frac{(r^{(k)})^T r^{(k)}}{(d^{(k)})^T Ad^{(k)}} \\
&x^{(k+1)} = x^{(k)} + t_k d^{(k)} \\
&r^{(k+1)} = r^{(k)} - t_k Ad^{(k)} \\
&(\text{note that } Ad^{(k)} \text{ is already at hand after computing } t_k) \\
&\beta_k = \frac{(r^{(k+1)})^T r^{(k+1)}}{(r^{(k)})^T r^{(k)}} \\
&d^{(k+1)} = r^{(k+1)} + \beta_k d^{(k)}
\end{aligned}$$

于是我们得到如下算法:

(共轭梯度法, 数值线性代数, 算法 5.2.1)

$$\begin{aligned}
&\overline{\text{Given positive definite matrix } A, \text{ vector } b \text{ and initial point } x^{(0)}} \\
&r^{(0)} = b - Ax^{(0)} \\
&d^{(0)} = r^{(0)} \\
&k = 0 \\
&\text{while } r^{(k)} \neq 0_n \quad (\rho^{(k)} \neq 0) \\
&\quad t_k = \frac{(r^{(k)})^T r^{(k)}}{(d^{(k)})^T Ad^{(k)}} \quad \left(\begin{cases} \rho^{(k)} = (r^{(k)})^T r^{(k)} \\ u^{(k)} = Ad^{(k)} \end{cases} ; t_k = \frac{\rho^{(k)}}{(d^{(k)})^T u^{(k)}} \right) \\
&\quad x^{(k+1)} = x^{(k)} + t_k d^{(k)} \\
&\quad r^{(k+1)} = r^{(k)} - t_k Ad^{(k)} \quad (r^{(k+1)} = r^{(k)} - t_k u^{(k)}) \\
&\quad \beta_k = \frac{(r^{(k+1)})^T r^{(k+1)}}{(r^{(k)})^T r^{(k)}} \quad (\beta_k = \frac{\rho^{(k+1)}}{\rho^{(k)}}) \\
&\quad d^{(k+1)} = r^{(k+1)} + \beta_k d^{(k)} \\
&\quad k = k + 1 \\
&\text{end} \\
&x = x^{(k)}
\end{aligned}$$

该算法每迭代一次仅需使用系数矩阵 A 做一次矩阵-向量运算 ($u^{(k)} = Ad^{(k)}$)

4.3.3 实用形式

数值线性代数 定理 5.2.1 表明, 在共轭梯度法中,

残差向量序列 $\{r^{(i)}\}_{i=0}^k$ 和下降方向向量序列 $\{d^{(i)}\}_{i=0}^k$ 分别是 Krylov 子空间 $\mathcal{K}(A, r^{(0)}, k+1)$ 的正交基和共轭正交基. 因此从理论上来说, 利用共轭梯度法最多 n 步便可得到方程组 $Ax = b$ 的精确解 $x^* = A^{-1}b$

它理论上是直接法, 但在实际计算中其有限步终止性并不成立.

这是由于误差的积累, 导致序列 $\{r^{(i)}\}_{i=0}^k$ 和 $\{d^{(i)}\}_{i=0}^k$ 随迭代次数增加而很快丧失其正交性.

因此我们将共轭梯度法作为一种迭代法使用,

而且通过设置 $\|r^{(k)}\|$ 的收敛阈值和最大迭代次数 $k_{\max}$ 来终止迭代.

(共轭梯度法的实用形式, 数值线性代数, 算法 5.3.1)

```
Given positive definite matrix  $A$ , vector  $b$  and initial point  $x^{(0)}$ 
 $x = x^{(0)}$ 
 $r = b - Ax$ 
 $d = r$ 
 $\rho = r^T r$ 
 $k = 0$ 
while ( $\sqrt{\rho} > \varepsilon \|b\|_2$ ) and ( $k < k_{\max}$ )
     $u = Ad$ 
     $t = \frac{\rho}{d^T u}$ 
     $x = x + td$ 
     $r = r - tu$ 
     $\tilde{\rho} = \rho$ 
     $\rho = r^T r$ 
     $\beta = \frac{\rho}{\tilde{\rho}}$ 
     $d = r + \beta d$ 
     $k = k + 1$ 
end
```

共轭梯度法作为一种实用的迭代法, 它主要有以下优点:

- 不需要预先估计任何参数
(区别于超松弛迭代法, 它需要估计最优松弛因子 ω_{opt})
- 每步迭代只需使用系数矩阵 A 做一次矩阵-向量运算 $u = Ad$
这不仅可以充分利用 A 的稀疏性,
而且适用于某些提供矩阵 A 显式形式较为困难, 但由已知向量 d 产生 $u = Ad$ 却十分方便的应用问题.
- 每步迭代主要进行的是向量之间的运算, 因此特别便于并行化.

4.3.4 收敛性分析

(数值线性代数, 定理 5.2.1)

考虑对称正定线性方程组 $Ax = b$

由共轭梯度法得到的残差向量序列 $\{r^{(i)}\}_{i=0}^k$ 和下降方向向量序列 $\{d^{(i)}\}_{i=0}^k$ 具有以下性质:

- $(d^{(i)})^T r^{(j)} = 0 \quad (0 \leq i < j \leq k)$
- $(r^{(i)})^T r^{(j)} = 0 \quad (0 \leq i \neq j \leq k)$
- $(d^{(i)})^T A d^{(j)} = 0 \quad (0 \leq i \neq j \leq k)$
- $\text{span}\{r^{(0)}, \dots, r^{(k)}\} = \text{span}\{d^{(0)}, \dots, d^{(k)}\} = \mathcal{K}(A, r^{(0)}, k+1)$
其中 Krylov 子空间 $\mathcal{K}(A, r^{(0)}, k+1) := \text{span}\{r^{(0)}, Ar^{(0)}, \dots, A^k r^{(0)}\}$

上述定理表明:

残差向量序列 $\{r^{(i)}\}_{i=0}^k$ 和下降方向向量序列 $\{d^{(i)}\}_{i=0}^k$ 分别是 Krylov 子空间 $\mathcal{K}(A, r^{(0)}, k+1)$ 的正交基和共轭正交基.

因此从理论上来说, 利用共轭梯度法最多 n 步便可得到方程组 $Ax = b$ 的精确解 $x^* = A^{-1}b$

它理论上是直接法, 但在实际计算中其有限步终止性并不成立.

这是由于误差的积累, 导致序列 $\{r^{(i)}\}_{i=0}^k$ 和 $\{d^{(i)}\}_{i=0}^k$ 随迭代次数增加而很快丧失其正交性.

回顾共轭梯度法的计算公式及其简化版本:

$$\begin{aligned}
& \overline{d^{(0)} = r^{(0)} = b - Ax^{(0)}} \\
& t_k = \frac{(r^{(k)})^T d^{(k)}}{(d^{(k)})^T Ad^{(k)}} = \frac{(r^{(k)})^T r^{(k)}}{(d^{(k)})^T Ad^{(k)}} \\
& x^{(k+1)} = x^{(k)} + t_k d^{(k)} \\
& r^{(k+1)} = b - Ax^{(k+1)} = r^{(k)} - t_k Ad^{(k)} \\
& \text{(note that } Ad^{(k)} \text{ is already at hand after computing } t_k) \\
& \beta_k = -\frac{(r^{(k+1)})^T Ad^{(k)}}{(d^{(k)})^T Ad^{(k)}} = \frac{(r^{(k+1)})^T r^{(k+1)}}{(r^{(k)})^T r^{(k)}} \\
& d^{(k+1)} = r^{(k+1)} + \beta_k d^{(k)}
\end{aligned}$$

用数学归纳法证明:

当 $k = 1$ 时, 我们有:

$$\begin{aligned}
& d^{(0)} = r^{(0)} = b - Ax^{(0)} \\
& t_0 = \frac{(r^{(0)})^T r^{(0)}}{(d^{(0)})^T Ad^{(0)}} \\
& x^{(1)} = x^{(0)} + t_0 d^{(0)} \\
& r^{(1)} = r^{(0)} - t_0 Ad^{(0)} \\
& \beta_0 = -\frac{(r^{(1)})^T Ad^{(0)}}{(d^{(0)})^T Ad^{(0)}} = \frac{(r^{(1)})^T r^{(1)}}{(r^{(0)})^T r^{(0)}} \\
& d^{(1)} = r^{(1)} + \beta_0 d^{(0)}
\end{aligned}$$

于是有:

$$(d^{(0)})^T r^{(1)} = (r^{(0)})^T r^{(1)} = (r^{(0)})^T (r^{(0)} - t_0 Ad^{(0)}) = (r^{(0)})^T r^{(0)} - \frac{(r^{(0)})^T r^{(0)}}{(d^{(0)})^T Ad^{(0)}} (r^{(0)})^T Ad^{(0)} = 0$$

$$(d^{(0)})^T Ad^{(1)} = (r^{(0)})^T A(r^{(1)} + \beta_0 d^{(0)}) = (r^{(0)})^T Ar^{(1)} - \frac{(r^{(1)})^T Ad^{(0)}}{(d^{(0)})^T Ad^{(0)}} (r^{(0)})^T Ad^{(0)} = 0$$

$$\text{span}\{d^{(0)}, d^{(1)}\} = \text{span}\{r^{(0)}, r^{(1)} + \beta_0 r^{(0)}\} = \text{span}\{r^{(0)}, r^{(1)}\} = \text{span}\{r^{(0)}, r^{(0)} - t_0 Ad^{(0)}\} = \text{span}\{r^{(0)}, Ar^{(0)}\}$$

因此命题对 $k = 1$ 的情况成立.

现在假设命题对 $k \geq 1$ 成立, 我们来证明其对 $k + 1$ 也成立.

- ① 要证明 $(d^{(i)})^T r^{(j)} = 0$ ($0 \leq i < j \leq k + 1$), 只要证明 $(d^{(i)})^T r^{(k+1)} = 0$ ($0 \leq i \leq k$):
对于 $i = k$ 的情况, 我们有:

$$\begin{aligned}
(d^{(k)})^T r^{(k+1)} &= (d^{(k)})^T (b - Ax^{(k+1)}) \\
&= (d^{(k)})^T [b - A(x^{(k)} + t_k d^{(k)})] \\
&= (d^{(k)})^T (r^{(k)} - t_k Ad^{(k)}) \\
&= (d^{(k)})^T r^{(k)} - \frac{(r^{(k)})^T d^{(k)}}{(d^{(k)})^T Ad^{(k)}} (d^{(k)})^T Ad^{(k)} \\
&= 0
\end{aligned}$$

对于 $0 \leq i \leq k - 1$ 的情况, 我们有:

$$\begin{aligned}
(d^{(i)})^T r^{(k+1)} &= (d^{(i)})^T (b - Ax^{(k+1)}) \\
&= (d^{(i)})^T [b - A(x^{(k)} + t_k d^{(k)})] \\
&= (d^{(i)})^T (r^{(k)} - t_k Ad^{(k)}) \\
&= (d^{(i)})^T r^{(k)} - \frac{(r^{(k)})^T d^{(k)}}{(d^{(k)})^T Ad^{(k)}} (d^{(i)})^T Ad^{(k)} \quad \left(\text{根据归纳假设} \begin{cases} (d^{(i)})^T r^{(k)} = 0 \\ (d^{(i)})^T Ad^{(k)} = 0 \end{cases} \right) \\
&= 0
\end{aligned}$$

- ② 由归纳假设可知 $\text{span}\{r^{(0)}, \dots, r^{(k)}\} = \text{span}\{d^{(0)}, \dots, d^{(k)}\}$
而由 ① 可知 $r^{(k+1)}$ 与 $d^{(0)}, \dots, d^{(k)}$ 正交, 因而 $r^{(k+1)}$ 也与 $r^{(0)}, \dots, r^{(k)}$ 正交,
即有 $(r^{(i)})^T r^{(k+1)} = 0$ ($0 \leq i \leq k$) 成立,
结合归纳假设 $(r^{(i)})^T r^{(j)} = 0$ ($0 \leq i \neq j \leq k$)
可知 $(r^{(i)})^T r^{(j)} = 0$ ($0 \leq i \neq j \leq k + 1$) 成立.

- ③ 要证明 $(d^{(i)})^T Ad^{(j)} = 0$ ($0 \leq i \neq j \leq k+1$), 只要证明 $\begin{cases} (d^{(i)})^T Ad^{(k+1)} = 0 & (0 \leq i \leq k-1) \\ (d^{(k+1)})^T Ad^{(k)} = 0 \end{cases}$

对于 $(d^{(i)})^T Ad^{(k+1)}$ ($0 \leq i \leq k-1$), 我们有:

$$\begin{aligned} (d^{(i)})^T Ad^{(k+1)} &= (d^{(i)})^T A(r^{(k+1)} + \beta_k d^{(k)}) \\ &= (d^{(i)})^T Ar^{(k+1)} + \beta_k (d^{(i)})^T Ad^{(k)} \quad (\text{根据归纳假设 } (d^{(i)})^T Ad^{(k)} = 0) \\ &= (d^{(i)})^T Ar^{(k+1)} + 0 \quad (\text{note that } r^{(i+1)} = r^{(i)} - t_i Ad^{(i)}) \\ &= \left[\frac{1}{t_i} (r^{(i+1)} - r^{(i)}) \right]^T r^{(k+1)} \\ &= \frac{1}{t_i} [(r^{(i+1)})^T r^{(k+1)} - (r^{(i)})^T r^{(k+1)}] \quad (\text{根据归纳假设 } (r^{(i+1)})^T r^{(k+1)} = (r^{(i)})^T r^{(k+1)} = 0) \\ &= \frac{1}{t_i} (0 - 0) \\ &= 0 \end{aligned}$$

对于 $(d^{(k+1)})^T Ad^{(k)}$, 我们有:

$$\begin{aligned} (d^{(k+1)})^T Ad^{(k)} &= (r^{(k+1)} + \beta_k d^{(k)})^T Ad^{(k)} \\ &= (r^{(k+1)})^T Ad^{(k)} + \beta_k (d^{(k)})^T Ad^{(k)} \\ &= (r^{(k+1)})^T Ad^{(k)} - \frac{(r^{(k+1)})^T Ad^{(k)}}{(d^{(k)})^T Ad^{(k)}} (d^{(k)})^T Ad^{(k)} \\ &= 0 \end{aligned}$$

- ④ 由归纳假设可知 $r^{(k)}, d^{(k)} \in \mathcal{K}(A, r^{(0)}, k+1) = \text{span}\{r^{(0)}, Ar^{(0)}, \dots, A^k r^{(0)}\}$

于是我们有:

$$\begin{aligned} r^{(k+1)} &= r^{(k)} - t_k Ad^{(k)} \in \mathcal{K}(A, r^{(0)}, k+2) = \text{span}\{r^{(0)}, Ar^{(0)}, \dots, A^k r^{(0)}, A^{k+1} r^{(0)}\} \\ d^{(k+1)} &= r^{(k+1)} + \beta_k d^{(k)} \in \mathcal{K}(A, r^{(0)}, k+2) = \text{span}\{r^{(0)}, Ar^{(0)}, \dots, A^k r^{(0)}, A^{k+1} r^{(0)}\} \end{aligned}$$

又注意到 ②③ 的结果表明:

向量组 $r^{(0)}, \dots, r^{(k)}, r^{(k+1)}$ 和 $d^{(0)}, \dots, d^{(k)}, d^{(k+1)}$ 都是线性无关的,

因此 $\text{span}\{r^{(0)}, \dots, r^{(k)}, r^{(k+1)}\} = \text{span}\{d^{(0)}, \dots, d^{(k)}, d^{(k+1)}\} = \mathcal{K}(A, r^{(0)}, k+2)$

综上所述, 定理得证.

(数值线性代数, 定理 5.2.2)

用共轭梯度法计算的近似解 $x^{(k)}$ 满足:

$$\begin{aligned} \varphi(x^{(k)}) &= \min\{\varphi(x) : x \in x^{(0)} + \mathcal{K}(A, r^{(0)}, k)\} \text{ where } \varphi(x) = x^T Ax - 2b^T x \\ \|x^{(k)} - x^*\|_A &= \min\{\|x - x^*\|_A : x \in x^{(0)} + \mathcal{K}(A, r^{(0)}, k)\} \end{aligned}$$

其中精确解 $x^* = A^{-1}b$, 而范数 $\|\cdot\|_A$ 的定义为 $\|x\|_A := \sqrt{x^T Ax}$

Krylov 子空间 $\mathcal{K}(A, r^{(0)}, k) = \{r^{(0)}, Ar^{(0)}, \dots, A^{k-1} r^{(0)}\}$

(共轭梯度法的收敛性, 数值线性代数, 定理 5.3.1)

考虑对称正定线性方程组 $Ax = b$, 将 A 分解为 $A = I + B$.

共轭梯度法至多迭代 $\text{rank}(B) + 1$ 步即可得到 $Ax = b$ 的精确解 $x^* = A^{-1}b$

- **上述定理表明:**

若系数矩阵 A 减去单位阵 I 得到的矩阵 B 的秩 $\text{rank}(B)$ 很小,

则共轭迭代法将会收敛得很快 (在 $\text{rank}(B) + 1$ 步内收敛).

其中 " $\text{rank}(B)$ 很小" 保证了共轭梯度法的残差向量序列 $\{r^{(i)}\}_{i=0}^k$ 的正交性还没有因误差积累而丧失.

- **证明:**

设初始向量为 $x^{(0)}$, 对应的残差向量为 $r^{(0)} = b - Ax^{(0)}$

注意到 $\text{rank}(B) = r$ 意味着对于任意 $k \geq 0$, Krylov 子空间 $\mathcal{K}(A, r^{(0)}, k+1)$ 的维度都不会超过 $r+1$.

$$\begin{aligned} \text{span}\{r^{(0)}, Ar^{(0)}, \dots, A^k r^{(0)}\} &= \text{span}\{r^{(0)}, (I+B)r^{(0)}, \dots, (I+B)^k r^{(0)}\} = \text{span}\{r^{(0)}, Br^{(0)}, \dots, B^k r^{(0)}\} \\ &\Rightarrow \end{aligned}$$

$$\dim(\text{span}\{r^{(0)}, Ar^{(0)}, \dots, A^k r^{(0)}\}) = \dim(\text{span}\{r^{(0)}, Br^{(0)}, \dots, B^k r^{(0)}\}) \leq \text{rank}(B) + 1$$

根据数值线性代数 定理 5.2.1 可知 $\text{span}\{r^{(0)}, \dots, r^{(k)}\} = \mathcal{K}(A, r^{(0)}, k+1)$

而且 $r^{(0)}, \dots, r^{(k)}$ 相互正交, 因此 $\dim(\mathcal{K}(A, r^{(0)}, k+1)) = \dim(\text{span}\{r^{(0)}, \dots, r^{(k)}\}) = k+1$

当共轭梯度法的迭代进行到第 $\text{rank}(B) + 1$ 步 (即 $k = \text{rank}(B)$) 时,

我们有 $\dim(\mathcal{K}(A, r^{(0)}, \text{rank}(B) + 1)) = \text{rank}(B) + 1$

于是一定有 $x^* = A^{-1}b \in \mathcal{K}(A, r^{(0)}, \text{rank}(B) + 1)$

再结合数值线性代数定理 5.2.2 可知 $\|x^{(k)} - x^*\|_A = \sqrt{(x^{(k)} - x^*)^T A (x^{(k)} - x^*)} = 0$, 即 $x^{(k)} = x^*$
命题得证.

(共轭梯度法的误差估计, 数值线性代数, 定理 5.3.2)

考虑对称正定线性方程组 $Ax = b$

共轭梯度法产生的序列 $\{x^{(k)}\}$ 满足:

$$\|x^{(k)} - x^*\|_A \leq 2 \left(\frac{\sqrt{\kappa_2(A)} - 1}{\sqrt{\kappa_2(A)} + 1} \right)^k \|x^{(0)} - x^*\|_A$$

其中精确解 $x^* = A^{-1}b$, 而范数 $\|\cdot\|_A$ 的定义为 $\|x\|_A := \sqrt{x^T A x}$

条件数 $\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$

- 上述定理给出的误差估计是十分粗糙的, 实际计算中其收敛速度往往比这个估计快得多.
不过它揭示了共轭梯度法的一个重要性质:
只要对称正定线性方程组 $Ax = b$ 的系数矩阵 A 是良态的 (即 $\kappa_2(A) \approx 1$), 共轭梯度法就会收敛得很快.
- 和最速下降法的收敛性分析一样, 这里也要使用 Chebyshev 多项式的性质.
第一类 Chebyshev 多项式 $T_n(x) = \cos(n \arccos(x))$

$$T_1(x) = \cos(\arccos(x)) = x$$

$$T_2(x) = \cos(2 \arccos(x)) = 2 \cos^2(\arccos(x)) - 1 = 2x^2 - 1$$

$$T_3(x) = \cos(3 \arccos(x)) = 4 \cos^3(\arccos(x)) - 3 \cos(\arccos(x)) = 4x^3 - 3x$$

$$2 \cos(x) \cos(nx) = \cos((n+1)x) + \cos((n-1)x)$$

$$2xT_n(x) = T_{n+1}(x) + T_{n-1}(x)$$

k 次 Chebyshev 多项式能在 $[-1, 1]$ 上最小化最大误差,

而在 $(-\infty, -1) \cup (1, \infty)$ 上的值超过任意一个次数不超过 k 的多项式.

(在 $[-1, 1]$ 上压缩程度最高, 在 $(-\infty, -1) \cup (1, \infty)$ 上增长速度最快)

- 收敛性分析表明: 共轭梯度法从本质上比最速下降法快.
- 在解线性方程组的 Krylov 子空间方法中,
特征值扎堆 (即重特征值多) 是好事情, 此时极小多项式次数较小.
这有利于共轭梯度法的收敛.
(其他 Krylov 子空间方法, 例如 GMRES 也是类似的, 但 GMRES 算法的收敛性比较麻烦)
而在特征值问题中, 特征值扎堆 (即重特征值多) 是坏事情.

4.3.5 预优共轭梯度法

考虑对称正定线性方程组 $Ax = b$

收敛性分析的结果表明:

当系数矩阵 A 只有少数几个互不相同的特征值或非常良态 (即 $\kappa_2(A) \approx 1$) 时, 共轭梯度法会收敛得非常快.

这启发我们在应用共轭梯度法时, 首先应设法将 $Ax = b$ 转化为一个等价方程组 $\tilde{A}\tilde{x} = \tilde{b}$,

使得新的系数矩阵 $\tilde{A}$ 只有少数几个互不相同的特征值或非常良态 (即 $\kappa_2(\tilde{A}) \approx 1$)

预优共轭梯度法正是基于这一基本思想产生的.

它通过选择一个对称正定阵 C 使得 $\tilde{A} = C^{-1}AC^{-1}$ 具有我们所希望的良好性质, 然后应用共轭梯度法.

其中我们记:

$$Ax = b \quad \Leftrightarrow \quad \tilde{A}\tilde{x} = \tilde{b}$$

$$\text{where } \tilde{A} = C^{-1}AC^{-1} \quad \tilde{x} = Cx \quad \tilde{b} = C^{-1}b$$

我们有以下计算公式 (其中 $\tilde{x}^{(0)}$ 为任意初始向量):

$$\tilde{d}^{(0)} = \tilde{r}^{(0)} = b - A\tilde{x}^{(0)}$$

$$\tilde{t}_k = \frac{(\tilde{r}^{(k)})^T \tilde{r}^{(k)}}{(\tilde{d}^{(k)})^T \tilde{A} \tilde{d}^{(k)}}$$

$$\tilde{x}^{(k+1)} = \tilde{x}^{(k)} + \tilde{t}_k \tilde{d}^{(k)}$$

$$\tilde{r}^{(k+1)} = \tilde{r}^{(k)} - \tilde{t}_k \tilde{A} \tilde{d}^{(k)}$$

(note that $\tilde{A} \tilde{d}^{(k)}$ is already at hand after computing $\tilde{t}_k$)

$$\tilde{\beta}_k = \frac{(\tilde{r}^{(k+1)})^T \tilde{r}^{(k+1)}}{(\tilde{r}^{(k)})^T \tilde{r}^{(k)}}$$

$$\tilde{d}^{(k+1)} = \tilde{r}^{(k+1)} + \tilde{\beta}_k \tilde{d}^{(k)}$$

按照上述公式迭代，我们需要事先计算 $\begin{cases} \tilde{A} = C^{-1}AC^{-1} \\ \tilde{b} = C^{-1}b \end{cases}$

最后还要将迭代得到的近似解 $\tilde{x}^{(k)}$ 变换回 $x^{(k)}$ ，即 $x^{(k)} = C^{-1}\tilde{x}^{(k)}$

实际上这些计算都是可以规避的。

记 $M = C^2$ ，并代入 $\begin{cases} x^{(k)} = C^{-1}\tilde{x}^{(k)} \\ \tilde{r}^{(k)} = \tilde{b} - \tilde{A}\tilde{x}^{(k)} = C^{-1}b - C^{-1}AC^{-1} \cdot Cx^{(k)} = C^{-1}(b - Ax^{(k)}) = C^{-1}r^{(k)} \text{ 即得:} \\ \tilde{d}^{(k)} = Cd^{(k)} \text{ (根据正交性结论推知)} \end{cases}$

$$r^{(0)} = b - Ax^{(0)}$$

$$d^{(0)} = z^{(0)} = M^{-1}r^{(0)}$$

$$t_k = \frac{(r^{(k)})^T z^{(k)}}{(d^{(k)})^T Ad^{(k)}}$$

$$x^{(k+1)} = x^{(k)} + t_k d^{(k)}$$

$$r^{(k+1)} = r^{(k)} - t_k Ad^{(k)}$$

(note that $Ad^{(k)}$ is already at hand after computing t_k)

$$z^{(k+1)} = M^{-1}r^{(k+1)}$$

$$\beta_k = \frac{(r^{(k+1)})^T z^{(k+1)}}{(r^{(k)})^T z^{(k)}}$$

$$d^{(k+1)} = z^{(k+1)} + \beta_k d^{(k)}$$

换言之，只需在有残量内积的地方将 $r^{(k)}$ 换为 $z^{(k)} = M^{-1}r^{(k)}$ 即可。

这样就得到了如下算法：

(预优共轭梯度法, 数值线性代数, 算法 5.4.1)

Given positive definite matrix A , vector b and initial point $x^{(0)}$

$$x = x^{(0)}$$

$$r = b - Ax$$

$$z = M^{-1}r \quad (\text{solve } Mz = r)$$

$$d = z$$

$$\rho = r^T z$$

$$k = 0$$

while $(\sqrt{r^T r} > \varepsilon \|b\|_2)$ and $(k < k_{\max})$

$$u = Ad$$

$$t = \frac{\rho}{d^T u}$$

$$x = x + td$$

$$r = r - tu$$

$$z = M^{-1}r \quad (\text{solve } Mz = r)$$

$$\tilde{\rho} = \rho$$

$$\rho = r^T z$$

$$\beta = \frac{\rho}{\tilde{\rho}}$$

$$d = z + \beta d$$

$$k = k + 1$$

end

我们称 $M = C^2$ 为**预优矩阵**，它是一个对称正定阵。

利用共轭梯度法的性质易知预优共轭梯度法具有如下性质：

- 残差向量 $\{r^{(i)}\}_{i=0}^k$ 是相互 M^{-1} 正交的，即 $(r^{(i)})^T M^{-1} r^{(j)} = 0 \quad (0 \leq i \neq j \leq k)$
- 方向向量 $\{d^{(i)}\}_{i=0}^k$ 是相互 A 正交的，即 $(d^{(i)})^T A d^{(j)} = 0 \quad (0 \leq i \neq j \leq k)$
- $(d^{(i)})^T r^{(j)} = 0 \quad (0 \leq i < j \leq k)$

- 近似解 $x^{(k)}$ 满足 $\|x^{(k)} - x^*\|_A \leq 2 \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} \right)^k \|x^{(0)} - x^*\|_A$
其中精确解 $x^* = A^{-1}b$ ，而范数 $\|\cdot\|_A$ 的定义为 $\|x\|_A := \sqrt{x^T A x}$
条件数 $\kappa = \kappa_2(M^{-\frac{1}{2}} A M^{-\frac{1}{2}}) = \frac{\lambda_{\max}(M^{-1}A)}{\lambda_{\min}(M^{-1}A)}$

4.3.6 邵老师的讲法

共轭梯度法优化的目标函数是 $\varphi(x) = \frac{1}{2} \|x - x_*\|_A^2 = \frac{1}{2} (x - x_*)^T A (x - x_*)$

我们希望找到一组 A -正交的下降方向 $\{d_k\}_{k=0}^n$ 使得 $x_* = x_0 + \sum_{k=0}^{n-1} t_k d_k$ (正交分解)

第 k 步近似解 x_k 为 A -内积意义下的残量最优解：

(与之对比，GMRES 是 l_2 范数意义下，即 Euclid 内积意义下)

$$x_k = x_0 + \sum_{i=0}^{k-1} t_i d_i = \arg \min_{x \in x_0 + K(A, r_0, k)} \|x - x_*\|_A$$

因此理论上共轭梯度法是有限终止的，其下降方向不仅是局部最优的，而且是全局最优的。

(实际应用中正交性会迅速失去，但依然能够收敛到不错的精度，因此共轭梯度法是少数不怕正交性损失的算法)

其中 $t_k d_k$ 为 A -内积意义下 $x_* - x_0$ 在 d_k 方向上的投影：

$$\begin{aligned} t_k d_k &= \frac{d_k d_k^T A}{d_k^T A d_k} (x_* - x_0) \\ &= d_k \cdot \frac{d_k^T r_0}{d_k^T A d_k} \end{aligned}$$

因此我们有 $t_k = \frac{d_k^T r_0}{d_k^T A d_k}$

于是最基础的迭代算法为：(A -内积意义下的 Gram-Schmidt 正交化)

$$\begin{aligned} d_0 &= r_0 \\ t_0 &= \frac{d_0^T r_0}{d_0^T A d_0} \\ x_1 &= x_0 + t_0 d_0 \\ r_1 &= b - A x_1 \\ d_1 &= r_1 - \frac{d_0 d_0^T A}{d_0^T A d_0} r_1 \\ t_1 &= \frac{d_1^T r_0}{d_1^T A d_1} \\ x_2 &= x_1 + t_1 d_1 \\ r_2 &= b - A x_2 \\ d_2 &= r_2 - \frac{d_0 d_0^T A}{d_0^T A d_0} r_2 - \frac{d_1 d_1^T A}{d_1^T A d_1} r_2 \\ t_2 &= \frac{d_2^T r_0}{d_2^T A d_2} \\ x_3 &= x_2 + t_2 d_2 \\ r_3 &= b - A x_3 \\ &\dots \end{aligned}$$

上述迭代格式简单但不实用。

其实用形式如下：(Gram-Schmidt 的长正交变成短正交)

$$d_0 = r_0 = b - Ax_0$$

loop:

$$\begin{aligned} t_k &= \frac{r_k^T r_k}{d_k^T A d_k} \\ x_{k+1} &= x_k + t_k d_k \\ r_{k+1} &= r_k - t_k A d_k \\ \beta_k &= \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k} \\ d_{k+1} &= r_{k+1} + \beta_k d_k \end{aligned}$$

上述迭代保证的数学性质:

$$\begin{aligned} r_{k+1}^T [r_0, \dots, r_k] &= [0, \dots, 0] \\ d_{k+1}^T A [d_0, \dots, d_k] &= [0, \dots, 0] \\ \mathcal{K}(A, r_0, k) &= \text{span}\{r_0, r_1, \dots, r_{k-1}\} \\ &= \text{span}\{d_0, d_1, \dots, d_{k-1}\} \\ \hline r_{k+1}^T [d_0, \dots, d_k] &= [0, \dots, 0] \\ d_{k+1}^T A [r_0, \dots, r_k] &= [0, \dots, 0] \end{aligned}$$

返回去考虑 A -内积意义下 Gram-Schmidt 过程的步长:

(当 $k = 0$ 时显然是一致的, 只需考虑 $k \geq 1$ 的情况即可)

$$\begin{aligned} t_k &= \frac{d_k^T A}{d_k^T A d_k} (x_* - x_0) \\ &= \frac{d_k^T r_0}{d_k^T A d_k} \\ &= \frac{(r_k + \beta_{k-1} d_{k-1})^T r_0}{d_k^T A d_k} \quad (\text{note that } r_k^T r_0 = 0 \text{ for all } k \geq 1) \\ &= \beta_{k-1} \frac{d_{k-1}^T r_0}{d_k^T A d_k} \\ &= \frac{r_k^T r_k}{r_{k-1}^T r_{k-1}} \frac{d_{k-1}^T r_0}{d_k^T A d_k} \end{aligned}$$

(归纳基础) 当 $k = 1$ 时, 我们有 $d_{k-1}^T r_0 = d_0^T r_0 = r_0^T r_0$

现假设 $d_{k-2}^T r_0 = r_{k-2}^T r_{k-2}$ ($k \geq 2$), 则我们有:

$$\begin{aligned} d_{k-1}^T r_0 &= (r_{k-1} + \beta_{k-2} d_{k-2})^T r_0 \quad (\text{note that } r_{k-1}^T r_0 = 0 \text{ for all } k \geq 2) \\ &= \beta_{k-2} d_{k-2}^T r_0 \quad (\text{note that } \beta_{k-2} = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k} \text{ and } d_{k-2}^T r_0 = r_{k-2}^T r_{k-2}) \\ &= \frac{r_{k-1}^T r_{k-1}}{r_{k-2}^T r_{k-2}} \cdot r_{k-2}^T r_{k-2} \\ &= r_{k-1}^T r_{k-1} \end{aligned}$$

这样我们就归纳地证明了 $d_{k-1}^T r_0 = r_{k-1}^T r_{k-1}$ ($\forall k \geq 1$)

于是我们有:

$$\begin{aligned} t_k &= \frac{r_k^T r_k}{r_{k-1}^T r_{k-1}} \frac{d_{k-1}^T r_0}{d_k^T A d_k} \\ &= \frac{r_k^T r_k}{r_{k-1}^T r_{k-1}} \frac{r_{k-1}^T r_{k-1}}{d_k^T A d_k} \\ &= \frac{r_k^T r_k}{d_k^T A d_k} \end{aligned}$$

这表明 A -内积的 Gram-Schmidt 过程中 t_k 的计算公式与共轭梯度法的实用形式的 t_k 计算公式是一致的。

4.4 Lanczos 方法

4.4.1 基本框架

(Rayleigh-Ritz 投影)

给定 Hermite 阵 $A \in \mathbb{C}^{n \times n}$

我们的目的就是寻找 $\mathbb{C}^n$ 的某一 k 维子空间的一组标准正交基 $Q \in \mathbb{C}^{n \times k}$

矩阵 A 在子空间 $\text{span}\{Q\}$ 上的正交投影就是 $Q^H A Q \in \mathbb{C}^{k \times k}$

这是一个小型稠密矩阵 (如果 $k \ll n$ 的话)

我们有很多方法 (例如 QR 算法、分而治之法) 得到其谱分解 $Q^H A Q = X M X^H$

其中 $M = \text{diag}\{\mu_1, \dots, \mu_k\}$, 而 $X = [x_1, \dots, x_k] \in \mathbb{C}^{k \times k}$ 为酉矩阵.

于是我们有:

$$\begin{aligned} Q^H A Q X &= X M \\ \Leftrightarrow \\ Q^H (A Q X - Q X M) &= 0_{k \times k} \end{aligned}$$

其中 M 的对角元 $\mu_1, \dots, \mu_k$ 就是 A 的近似特征值

而 $Y := Q X = [Q x_1, \dots, Q x_k] \in \mathbb{C}^{n \times k}$ 的列向量 $y_1, \dots, y_k$ 就是 A 的近似特征向量.

(其中 $y_i = Q x_i$ ($i = 1, \dots, k$))

我们称 $(\mu_1, y_1), \dots, (\mu_k, y_k)$ 为 **Ritz 对**.

依照不同方式构建子空间及其标准正交基, 就可以得到不同的算法.

值得注意的是, $Q \in \mathbb{C}^{n \times k}$ 必须是列标准正交.

否则基于 $Q^H A Q = X M X^H$ 我们只能得到广义特征值问题 $Q^H (A Q X - Q (Q^H Q)^{-1} X M)$

(Hermite 特征值问题的扰动分析)

设 (λ, x) 是 Hermite 阵 $A \in \mathbb{C}^{n \times n}$ 的特征对 (即满足 $Ax = x\lambda$)

考虑近似特征向量 $\hat{x} = x + \Delta x$ 和近似特征值 (即对应的 Rayleigh 商) $\hat{\lambda} = \frac{\hat{x}^H A \hat{x}}{\hat{x}^H \hat{x}}$

于是我们有:

$$\begin{aligned} \hat{\lambda} - \lambda &= \frac{\hat{x}^H A \hat{x}}{\hat{x}^H \hat{x}} - \lambda \\ &= \frac{\hat{x}^H (A - \lambda I) \hat{x}}{\hat{x}^H \hat{x}} \\ &= \frac{(x + \Delta x)^H (A - \lambda I) (x + \Delta x)}{\hat{x}^H \hat{x}} \quad \left(\text{note that } \begin{cases} (A - \lambda I)x = Ax - x\lambda = 0_n \\ x^H (A - \lambda I) = x^H A - \lambda x^H = 0_n^T \end{cases} \right) \\ &= \frac{\Delta x^H (A - \lambda I) \Delta x}{\hat{x}^H \hat{x}} \end{aligned}$$

因此向前误差 $\Delta\lambda = \hat{\lambda} - \lambda$ 满足 $|\Delta\lambda| = O(\|\Delta x\|_2^2)$

这展示了 Hermite 特征值问题的一个非常好的性质:

特征值的收敛速度 (精度) 是特征向量的收敛速度 (精度) 的两倍.

例如物理学中计算谱的时候, 即使特征向量的精度没有那么多高, 特征值也能很快达到足够的精度 (如化学精度)

Tips: 向前误差是计算解与精确解的误差.

向后误差是作用在原始数据上的能使得计算解成为 "精确解" 的扰动 (它可能不存在, 也可能有多个)

向前误差是条件数与向后误差的共同作用.

4.4.2 基础算法

Lanczos 方法可以用来求解大规模稀疏对称特征值问题 $Ax = x\lambda$.

该方法对给定的矩阵 A 进行局部三对角化 (算法过程中的子矩阵也是稀疏的)

同时 A 两端的特征值可以很快收敛出来.

因此在只需要对称阵 A 的若干个最大或最小特征值的时候, Lanczos 算法有明显的优越性.

我们复述 FDU 数值算法 2. 稠密线性最小二乘问题的解法.md 中有关 Lanczos 过程的内容.

给定对称阵 $A \in \mathbb{R}^{n \times n}$ 和单位向量 $q_1 \in \mathbb{R}^n$

我们记:

$$\tilde{T}_k = \begin{bmatrix} \alpha_1 & \beta_1 & & & \\ \beta_1 & \alpha_2 & \beta_2 & & \\ & \beta_2 & \alpha_3 & \ddots & \\ & & \ddots & \ddots & \beta_{k-1} \\ & & & \beta_{k-1} & \alpha_k \\ & & & & \beta_k \end{bmatrix}$$

记 $Q_k := [q_1, \dots, q_k]$ (其中 $1 \leq k \leq \text{rank}(\mathcal{K}(A, q_1, n))$)

根据 $AQ_k = Q_{k+1}\tilde{T}_k$ 可知 $Aq_k = \beta_{k-1}q_{k-1} + \alpha_k q_k + \beta_k q_{k+1}$

由于 $q_1, \dots, q_{k+1}$ 标准正交, 故我们有 $q_k^T Aq_k = \alpha_k$

最后有 $\beta_k q_{k+1} = Aq_k - \alpha_k q_k - \beta_{k-1} q_{k-1}$

(对于 $A \in \mathbb{C}^{n \times n}$ 是 Hermite 阵的情况, 推理过程基本相同, 参见 Homework 13 Problem 4)

(Lanczos 过程, Matrix Computation 9.1.2 节)

Given symmetric matrix $A \in \mathbb{R}^{n \times n}$ and $q_1 \in \mathbb{R}^n$ such that $\|q_1\|_2 = 1$

$k = 0$; $\beta_0 = 1$; $q_0 = 0_n$; $r_0 = q_1$

while $\beta_k \neq 0$

$$q_{k+1} = \frac{r_k}{\beta_k}$$

$$k = k + 1$$

$$\alpha_k = q_k^T Aq_k$$

$$r_k = Aq_k - \alpha_k q_k - \beta_{k-1} q_{k-1}$$

$$\beta_k = \|r_k\|_2$$

end

(Matrix Computation 定理 9.1.1)

设 $A \in \mathbb{R}^{n \times n}$ 为对称阵, $q_1 \in \mathbb{R}^n$ 满足 $\|q_1\|_2 = 1$

则 Lanczos 迭代进行到第 $k = \text{rank}(\mathcal{K}(A, q_1, n))$ 步终止.

此外, 对于任意 $k = 1, \dots, \text{rank}(\mathcal{K}(A, q_1, n))$ 我们都有:

$$\begin{aligned} AQ_k &= A[q_1, \dots, q_k] \\ &= [q_1, \dots, q_k, q_{k+1}] \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{k-1} \\ & & \beta_{k-1} & \alpha_k \\ & & & \beta_k \end{bmatrix} \\ &= Q_{k+1}\tilde{T}_k \\ &= [Q_k, q_{k+1}] \begin{bmatrix} T_k \\ \beta_k e_k^T \end{bmatrix} \\ &= Q_k T_k + \beta_k q_{k+1} e_k^T \\ &= Q_k T_k + r_k e_k^T \end{aligned}$$

其中 $q_1, \dots, q_k, q_{k+1}$ 标准正交, 且 $\text{Range}(Q_k) = \mathcal{K}(A, q_1, k) = \text{span}\{q_1, Aq_1, \dots, A^{k-1}q_1\}$

而 $e_k \in \mathbb{R}^k$ 代表 $\mathbb{R}^k$ 的第 k 个标准正交基向量.

在 Lanczos 过程中, $\beta_k = 0$ 是最受欢迎的, 因为这说明找到一个不变子空间.

但在实际计算中, 上述情况很少发生.

幸运的是, 我们可以证明 T_k 的最大、最小特征值是 A 的最大、最小特征值的极好的近似.

(Matrix Computation 定理 9.1.2)

设 Lanczos 迭代已经进行了 k 步, 得到了 $AQ_k = Q_k T_k + r_k e_k^T$

设对称三对角阵 $T_k \in \mathbb{R}^{k \times k}$ 的谱分解为 $T_k = X_k M_k X_k^T$

其中 $X_k = [x_{ij}] \in \mathbb{R}^{k \times k}$ 为实正交阵, $M_k = \text{diag}\{\mu_1, \dots, \mu_k\}$ (特征值按非增次序排列 $\mu_1 \geq \dots \geq \mu_k$)

记 $Y_k = [y_1, \dots, y_k] = Q_k X_k \in \mathbb{R}^{n \times k}$ (显然列标准正交)

则我们有:

$$\|Ay_i - y_i \mu_i\|_2 = |\beta_k| \cdot |x_{ki}| \quad (i = 1, \dots, k)$$

我们称 (μ_i, y_i) 是 Krylov 子空间 $\text{Range}(Q_k) = \mathcal{K}(A, q_1, k)$ 的 Ritz 对.

- **推论:** (T_k 逼近 A 特征值的误差界)

$$\min_{\lambda \in \text{eig}(A)} |\mu_i - \lambda| \leq |\beta_k| |x_{ki}| \quad (i = 1, \dots, k)$$

- **证明:**

对 $AQ_k = Q_k T_k + r_k e_k^T$ 左右同乘 X_k 即得:

$$\begin{aligned} AY_k &= AQ_k X_k \\ &= (Q_k T_k + r_k e_k^T) X_k \\ &= Q_k T_k X_k + r_k e_k^T X_k \\ &= Q_k X_k M_k + r_k e_k^T X_k \\ &= Y_k M_k + r_k e_k^T M_k \end{aligned}$$

于是我们有:

$$\begin{aligned} [Ay_1, \dots, Ay_k] &= A[y_1, \dots, y_k] \\ &= AY_k \\ &= Y_k M_k + r_k e_k^T X_k \\ &= [y_1, \dots, y_k] \begin{bmatrix} \mu_1 & & \\ & \ddots & \\ & & \mu_k \end{bmatrix} + r_k [x_{k1}, \dots, x_{kk}] \\ &= [y_1 \mu_1 + r_k x_{k1}, \dots, y_k \mu_k + r_k x_{kk}] \end{aligned}$$

因此对于任意 $i = 1, \dots, k$ 我们都有:

$$Ay_i = y_i \mu_i + r_k x_{ki}$$

从而有:

$$\begin{aligned} \|Ay_i - y_i \mu_i\|_2 &= \|r_k x_{ki}\|_2 \\ &= \|\beta_k q_{k+1} x_{ki}\|_2 \\ &= |\beta_k| \cdot |x_{ki}| \cdot \|q_{k+1}\|_2 \quad (i = 1, \dots, k) \\ &= |\beta_k| \cdot |x_{ki}| \end{aligned}$$

命题得证.

邵老师的讲法: (记号可能有冲突, x 与 y 的意义反了)

给定 Hermite 正定阵 $A \in \mathbb{C}^{n \times n}$, 设其特征值为 $0 < \lambda_1 \leq \dots \leq \lambda_n$

我们想要计算其中最大 (或最小) 的几个特征值 (边上的特征值好算, 中间的特征值不好算)

给定初始向量 $x_0 \in \mathbb{C}^n$

乘幂法的过程 $x_0, Ax_0, \dots, A^{k-1}x_0$ 就是落在 Krylov 子空间中的.

我们贪心地在 Krylov 子空间中找最好的近似特征向量.

设第 k 步迭代根据 Lanczos 过程得到:

$$\begin{aligned} AQ_k &= A[q_1, \dots, q_k] \\ &= [q_1, \dots, q_k, q_{k+1}] \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{k-1} \\ & & \beta_{k-1} & \alpha_k \\ & & & \beta_k \end{bmatrix} \\ &= Q_{k+1} \tilde{T}_k \\ &= [Q_k, q_{k+1}] \begin{bmatrix} T_k \\ \beta_k e_k^T \end{bmatrix} \\ &= Q_k T_k + \beta_k q_{k+1} e_k^T \\ &= Q_k T_k + r_k e_k^T \end{aligned}$$

其中 $q_1 = \frac{x_0}{\|x_0\|_2}$

注意到第 k 个近似特征向量 $x_k \in \mathcal{K}(A, x_0, k) = \text{span}\{Q_k\}$

因此我们可以设 $x_k = Q_k y_k$, 我们希望它满足:

$$\begin{aligned} Ax_k &\approx x_k \lambda \\ \Leftrightarrow \\ AQ_k y_k &\approx Q_k y_k \lambda \end{aligned}$$

其中 λ 是 A 的某个特征值.

设 μ_k 是特征值 λ 的近似, 我们定义残量 $r_k = AQ_k y_k - Q_k y_k \mu_k$

Galerkin 条件 (即残量正交化条件) 要求 $r_k \perp \mathcal{K}(A, x_0, k)$, 即等价于要求:

$$\begin{aligned} Q_k^H r_k &= Q_k^H (AQ_k y_k - Q_k y_k \mu_k) \\ &= Q_k^H A Q_k y_k - Q_k^H Q_k y_k \mu_k \\ &= T_k y_k - y_k \mu_k \\ &= 0_k \end{aligned}$$

(这与 Rayleigh-Ritz 迭代格式很相似, 不过 Lanczos 过程保证了 $Q_k^H A Q_k = T_k$)

因此通过对称隐式 QR 算法 (当 k 较大时可以使用分而治之法) 求解子问题 $T_k y_k = y_k \mu_k$,

我们可以得到 A 的近似特征值 μ_k 和近似特征向量 $x_k = Q_k y_k$

Cauchy 交错定理保证了 T_{k+1} 的特征值相比 T_k 的特征值向外扩张.

有的算法往大的方向去, 有的算法往小的方向去.

因此我们可以计算 A 的最大 (或最小) 的几个特征值 (边上的特征值好算, 中间的特征值不好算)

Cauchy 交错定理 (Cauchy Interlacing Theorem) 描述了 n 阶 Hermite 阵与其 $n-1$ 阶主子阵的特征值关系.

(Cauchy 交错定理, Matrix Analysis 定理 4.3.17)

给定 Hermite 阵 $A \in \mathbb{C}^{n \times n}$, 特征值按非减的次序排列: $\lambda_1(A) \leq \dots \leq \lambda_n(A)$

考虑 A 的 $n-1$ 主子阵 $B = A_{(1:n-1, 1:n-1)} \in \mathbb{C}^{(n-1) \times (n-1)}$, 并记 $A = \begin{bmatrix} B & y \\ y^H & a \end{bmatrix}$

特征值按非减的次序排列: $\lambda_1(B) \leq \dots \leq \lambda_{n-1}(B)$

则我们有如下的交错性质:

$$\begin{aligned} \lambda_1(A) \leq \lambda_1(B) \leq \lambda_2(A) \leq \dots \leq \lambda_{n-1}(A) \leq \lambda_{n-1}(B) \leq \lambda_n(A) \\ \Leftrightarrow \\ \lambda_i(A) \leq \lambda_i(B) \leq \lambda_{i+1}(A) \quad (\forall i = 1, \dots, n-1) \end{aligned}$$

其中 $\lambda_i(A) = \lambda_i(B)$ 成立当且仅当存在非零向量 $z \in \mathbb{C}^{n-1}$ 使得 $\begin{cases} Bz = z\lambda_i(B) \\ y^H z = 0 \end{cases}$

而 $\lambda_i(B) = \lambda_{i+1}(A)$ 成立当且仅当存在非零向量 $z \in \mathbb{C}^{n-1}$ 使得 $\begin{cases} Bz = z\lambda_{i+1}(A) \\ y^H z = 0 \end{cases}$

若 B 没有与 y 正交的特征向量, 则上述不等式均为严格不等式.

理论上, 算法在第 n 步停止.

但 Lanczos 向量的正交性会很快丧失, 因此数值上是不稳定的.

Paige 挽救了这个算法: 当正交性损失到一定程度时, 会有一个特征值收敛.

对于非对称矩阵就是使用 Arnoldi 方法寻找 Krylov 子空间的正交基:

$$AQ_k = Q_{k+1} \tilde{H}_k = Q_k H_k + h_{k+1,k} q_{k+1}$$

子问题相应的变为求解 H_k 的特征值和特征向量 (我们可以使用 Francis 双步位移的隐式 QR 算法)

基础思想仍是将大问题变为小问题来解决.

4.4.3 收敛性

(Kaniel-Paige 收敛性理论, Matrix Computation 定理 9.1.3)

设 $A \in \mathbb{R}^{n \times n}$ 是实对称阵, 特征值为 $\lambda_1 \geq \dots \geq \lambda_n$, 对应标准特征向量为 $x_1, \dots, x_n$

设 Lanczos 迭代已经进行了 k 步, 得到 $AQ_k = Q_k T_k + r_k e_k^T$

其中 $Q_k = [q_1, \dots, q_k] \in \mathbb{R}^{n \times k}$ 列标准正交,

而 $T_k \in \mathbb{R}^{k \times k}$ 为对称三对角阵, 特征值为 $\mu_1 \geq \dots \geq \mu_k$

则我们有:

$$\lambda_1 \geq \mu_1 \geq \lambda_1 - \frac{\lambda_1 - \lambda_n}{(f_{k-1}(1 + 2\rho_1))^2} \tan^2(\theta_1)$$

其中 $\cos(\theta_1) = |q_1^T x_1|$ 是 q_1 和 x_1 之间的夹角余弦, $\rho_1 = \frac{\lambda_1 - \lambda_2}{\lambda_2 - \lambda_n}$
而 $f_m(t) = \cos(m \arccos(t))$ ($t \in [-1, 1]$) 为第一类 Chebyshev 多项式.

- 用 $-A$ 替换 A 就得到: (原书中有记号错误)
(Matrix Computation 推论 9.1.4)

$$\lambda_n \leq \mu_k \leq \lambda_n + \frac{\lambda_1 - \lambda_n}{(f_{k-1}(1 + 2\rho_n))^2} \tan^2(\theta_n)$$

其中 $\cos(\theta_n) = |q_k^T x_n|$ 是 q_k 和 x_n 之间的夹角余弦, $\rho_n = \frac{\lambda_{n-1} - \lambda_n}{\lambda_1 - \lambda_{n-1}}$

4.4.4 完全重正交化

正交性损失主要是由相消引起的, 而不是舍入误差的累积结果 (这和经典 Gram-Schmidt 正交化很像)
为避免正交性损失, 我们将新计算的 Lanczos 向量 q_{k+1} 与之前计算的 Lanczos 向量 $q_1, \dots, q_k$ 正交化.
这就得到了第一个 "实用" 的 Lanczos 算法.

给定 $r_0, r_1, \dots, r_{k-1} \in \mathbb{R}^n$, 并假定已计算出 Householder 矩阵 $H_0, H_1, \dots, H_{k-1}$ 使得 $(H_0 H_1 \cdots H_{k-1})^T [r_0, r_1, \dots, r_{k-1}]$ 为上三角形矩阵. 用 $[q_1, \dots, q_k]$ 表示 Householder 矩阵乘积 $(H_0 H_1 \cdots H_{k-1})$ 的前 k 列. 现假定给出向量 $r_k \in \mathbb{R}^n$, 我们希望在下述方向上得到一单位向量 q_{k+1} :

$$w = r_k - \sum_{i=1}^k (q_i^T r_k) q_i \in \text{span}\{q_1, \dots, q_k\}^\perp.$$

如果选取 Householder 矩阵 H_k 使得 $(H_0 H_1 \cdots H_k)^T [r_0, r_1, \dots, r_k]$ 为上三角形矩阵, 则 $H_0 H_1 \cdots H_k$ 的第 $k+1$ 列就是我们所需要单位向量.

将上述 Householder 计算和 Lanczos 算法结合起来,
我们就可得到与机器精度无关的 Lanczos 向量.

(完全重正交化的 Lanczos 过程, Matrix Computation 9.2.3 节)

Given symmetric matrix $A \in \mathbb{R}^{n \times n}$ and $q_1 \in \mathbb{R}^n$ such that $\|q_1\|_2 = 1$
 $r_0 = q_1$
 Determine Householder matrix H_0 such that $H_0 r_0 = \|r_0\|_2 e_1 = e_1$
 $\alpha_1 = q_1^T A q_1$
 for $k = 1 : n - 1$
 $r_k = A q_k - \alpha_k q_k - \beta_{k-1} q_{k-1}$
 $w = (H_{k-1} \cdots H_0) r_k$ (denote $w = [w_1, \dots, w_k, w_{k+1}, w_{k+2}, \dots, w_n]^T$)
 Determine Householder matrix H_k such that $H_k w = [w_1, \dots, w_k, \beta_k, 0, \dots, 0]^T$
 $q_{k+1} = [(H_0 \cdots H_{k-1}) H_k] e_{k+1}$
 $\alpha_{k+1} = q_{k+1}^T A q_{k+1}$
 end

实际应用中, 我们只存储 Householder 向量 v_k
(其中 $H_k = I_n - 2v_k v_k^H$, 不过注意 v_k 的前 k 个元素为零, 因此只需存储后 $n - k$ 个分量即可)
而且我们没有必要去计算 $w = (H_{k-1} \cdots H_0) r_k$ 的前 k 个分量,
因为第 k 步 Householder 矩阵的计算是针对 w 的后 $n - k$ 个分量进行的.

不幸的是, 在完全重正交化的计算中, 这些措施的意义并不大.
因为在 Lanczos 算法的第 k 步, 计算 Householder 矩阵会增加 $O(kn)$ 的计算量.
计算 q_{k+1} 时也要用到对应于 $H_0, \dots, H_{k-1}, H_k$ 的 Householder 向量.
当 n 和 k 很大时, 完全重正交化的代价是我们难以接受的.

4.4.5 有选择的重正交化

Paige 误差分析的一个惊人的结论是:

Lanczos 向量正交性的丢失与 Ritz 对的收敛性是密不可分的.

具体来说, 设对 $\hat{T}_k$ 应用对称隐式 QR 算法得到 Ritz 值的计算解 $\hat{\mu}_1, \dots, \hat{\mu}_k$

以及一个由特征向量构成的近似正交的矩阵 $\hat{X}_k = [\hat{x}_{ij}] \in \mathbb{C}^{k \times k}$

设第 k 步得到的 Lanczos 向量构成的矩阵为 $\hat{Q}_k$, 新计算出的 Lanczos 向量为 $\hat{q}_{k+1}$

对称三对角阵 $\hat{T}_k$ 的次对角线元素为 $\hat{\beta}_1, \dots, \hat{\beta}_{k-1}$, 新计算出的次对角元为 $\hat{\beta}_k$

计算 Ritz 向量构成的矩阵 $\hat{Y}_k = [\hat{y}_1, \dots, \hat{y}_k] = \text{fl}(\hat{Q}_k \hat{X}_k)$

可以证明对于任意 $i = 1, \dots, k$ 都有:

$$\begin{aligned} \|A\hat{y}_i - \hat{\mu}_i \hat{y}_i\|_2 &\approx |\hat{\beta}_k| |\hat{x}_{ki}| \\ |\hat{q}_{k+1}^T \hat{y}_i| &\approx \frac{\text{eps} \|A\|_2}{|\hat{\beta}_k| |\hat{x}_{ki}|} \end{aligned} \quad (1)$$

换言之, 新计算出的 Lanczos 向量 $\hat{q}_{k+1}$ 倾向于在任何已经收敛的 Ritz 向量的方向上有不可忽视的非零分量.

因此我们不必将 $\hat{q}_{k+1}$ 与之前计算出的所有 Lanczos 向量正交化,

只需让它与收敛的 Ritz 向量构成的集合正交化, 就可以达到同样的效果.

(即将它投影到收敛的近似特征向量 (Ritz 向量) 的正交补空间)

我们称一个 Ritz 对 $(\hat{\mu}, \hat{y})$ 是 "好的", 如果它满足:

$$\|A\hat{y} - \hat{y}\hat{\mu}\|_2 \approx \sqrt{\text{eps}} \|A\|_2$$

通常来说, 好的 Ritz 向量的数量要比 Lanczos 向量少很多.

我们可以在每步计算完 $\hat{T}_k$ 的谱分解之后, 基于 (1) 式根据 $|\hat{x}_{ki}|$ ($i = 1, \dots, k$) 来挑选 "好的" Ritz 向量.

一种更有效的方式是用以下结果来估计正交性损失 $\|I_k - \hat{Q}_k^T \hat{Q}_k\|_2$:

(Matrix Computation 引理 9.2.1)

设 $Q_+ = [Q, q]$ (其中 $Q \in \mathbb{R}^{n \times k}$ 近似列标准正交, $q \in \mathbb{R}^n$ 是近似单位向量, 且近似正交于 Q 的列空间)

若 $\|I_k - Q^T Q\|_2 \leq \varepsilon$ 且 $|1 - q^T q| \leq \delta$, 则我们有:

$$\begin{aligned} \|I_{k+1} - Q_+^T Q_+\|_2 &\leq \varepsilon_+ \\ \text{where } \varepsilon_+ &= \frac{1}{2}(\varepsilon + \delta + \sqrt{(\varepsilon - \delta)^2 + 4\|Q^T q\|_2^2}) \end{aligned}$$

记 $\hat{Q}_{k+1} = [\hat{Q}_k, \hat{q}_{k+1}]$

假设 $\hat{q}_{k+1}$ 已经与当前的 "好的" Ritz 向量集正交化并标准化了 (使得 $|1 - \hat{q}_{k+1}^T \hat{q}_{k+1}| \leq \delta \approx \text{eps}$)

当我们已知 $\|I_k - \hat{Q}_k^T \hat{Q}_k\|_2$ 的界时, 就可通过上述引理得到 $\|I_{k+1} - \hat{Q}_{k+1}^T \hat{Q}_{k+1}\|_2$ 的界.

这一简单的计算过程不需要 $\hat{q}_1, \dots, \hat{q}_k$ 的参与, 因此开销是很小的.

如果当前的估计界显示已失去正交性时, 就说明可能有更多的 Ritz 向量收敛了.

此时需要考虑扩大 "好的" Ritz 向量的集合, 之后再求解子问题 (即将 $\hat{T}_k$ 对角化)

这样我们就得到了实用的 Lanczos 算法.

4.4.6 应用于奇异值

回忆起 SVD 迭代的前端是双对角化 (无论是上双对角化还是下双对角化都可以)

设 $[U, U_\perp]^H A V = \begin{bmatrix} B \\ 0_{(m-n) \times n} \end{bmatrix}$ 是矩阵 $A \in \mathbb{C}^{m \times n}$ ($m \geq n$) 的上双对角化:

$$\begin{aligned} U &= [u_1, \dots, u_n] \in \mathbb{C}^{m \times n} \text{ such that } U^T U = I_n \\ U_\perp &= [u_{n+1}, \dots, u_m] \in \mathbb{C}^{m \times (m-n)} \text{ such that } U_\perp^T U_\perp = I_{m-n} \\ V &= [v_1, \dots, v_n] \in \mathbb{C}^{n \times n} \text{ such that } V^T V = I_n \\ B &= \begin{bmatrix} \alpha_1 & \beta_1 & & & \\ & \alpha_2 & \beta_2 & & \\ & & \alpha_3 & \ddots & \\ & & & \ddots & \beta_{n-1} \\ & & & & \alpha_n \end{bmatrix} \end{aligned}$$

其精简形式为 $U^H AV = B$

其中 $\alpha_1, \dots, \alpha_n$ 和 $\beta_1, \dots, \beta_{n-1}$ 均为实数.

如果 A 是小型稠密矩阵, 则我们可用行列交替的 Householder 变换将其双对角化.

但当 A 是大型稀疏矩阵时, 我们必须发展直接计算 B 的方法.

(因为 Householder 双对角化过程中会出现大型稠密子矩阵)

受 Lanczos 算法的启发, 我们比较下列矩阵方程的列:

$$\begin{aligned} [Av_1, Av_2, \dots, Av_n] &= AV = UB = [\alpha_1 u_1, \beta_1 u_1 + \alpha_2 u_2, \dots, \beta_{n-1} u_{n-1} + \alpha_n u_n] \\ [A^H u_1, \dots, A^H u_{n-1}, A^H u_n] &= A^H U = VB^H = [\alpha_1 v_1 + \beta_1 v_2, \dots, \alpha_{n-1} v_{n-1} + \beta_{n-1} v_n, \alpha_n v_n] \end{aligned}$$

定义 $\beta_0 u_0 = 0$ 和 $\beta_n v_{n+1} = 0$, 则我们有如下迭代格式:

$$\begin{cases} Av_k = \beta_{k-1} u_{k-1} + \alpha_k u_k \\ A^H u_k = \alpha_k v_k + \beta_k v_{k+1} \end{cases} \quad (k = 1, \dots, n)$$

我们记 $\begin{cases} r_k = Av_k - \beta_{k-1} u_{k-1} = \alpha_k u_k \\ p_k = A^H u_k - \alpha_k v_k = \beta_k v_{k+1} \end{cases}$

根据 U, V 的列标准正交性可知:

$$\begin{cases} \alpha_k = \pm \|r_k\|_2 \\ u_k = \frac{r_k}{\alpha_k} \\ \beta_k = \pm \|p_k\|_2 \\ v_k = \frac{p_k}{\beta_k} \end{cases}$$

恰当地安排它们的顺序, 就得到如下算法:

(Lanczos 上双对角化算法 (Golub-Kahan), Matrix Computation 9.3.3 节)

```

Given  $A \in \mathbb{C}^{m \times n}$  and  $v_1 \in \mathbb{C}^n$  such that  $\|v_1\|_2 = 1$ 
 $\beta_0 = 1; u_0 = 0_n; p_0 = v_1;$ 
 $k = 0;$ 
while  $\beta_k \neq 0$ 
     $v_k = \frac{p_k}{\beta_k}$ 
     $k = k + 1$ 
     $r_k = Av_k - \beta_{k-1} u_{k-1}$ 
     $\alpha_k = \|r_k\|_2$ 
     $u_k = \frac{r_k}{\alpha_k}$ 
     $p_k = A^H u_k - \alpha_k v_k$ 
     $\beta_k = \|p_k\|_2$ 
end
    
```

其迭代格式可以等价地表示为:

$$\begin{aligned} AV_k &= [Av_1, \dots, Av_k] = [u_1, \dots, u_k] \begin{bmatrix} \alpha_1 & \beta_1 & & \\ & \alpha_3 & \ddots & \\ & & \ddots & \beta_{k-1} \\ & & & \alpha_k \end{bmatrix} = U_k B_k \\ A^H U_k &= [A^H u_1, \dots, A^H u_k] = [v_1, \dots, v_k, v_{k+1}] \begin{bmatrix} \alpha_1 & & & \\ \beta_1 & \alpha_3 & & \\ & \ddots & \ddots & \\ & & \beta_{k-1} & \alpha_k \\ \hline & & & \beta_k \end{bmatrix} = V_{k+1} \tilde{B}_k^H \end{aligned}$$

上述算法以 v_1 为初始向量 (作用在 A 上开启迭代), 且新的 β_k, v_{k+1} 是由 A^H 产生的.

邵老师提供的 insight:

Lanczos 双对角化算法在本质上等价于对以下 Hermite 阵应用 Lanczos 算法:

$$\tilde{A} = \begin{bmatrix} 0 & A \\ A^H & 0 \end{bmatrix}$$

(Matrix Analysis 定理 7.3.3)

给定复矩阵 $A \in \mathbb{C}^{m \times n}$, 记 $q := \min\{m, n\}$

设 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_q$ 为 A 的奇异值

定义 Hermite 阵 $\tilde{A} = \begin{bmatrix} 0_{m \times m} & A \\ A^H & 0_{n \times n} \end{bmatrix} \in \mathbb{C}^{(m+n) \times (m+n)}$

则 $\tilde{A}$ 的特征值为 $-\sigma_1 \leq \dots \leq -\sigma_q \leq \underbrace{0 \dots 0}_{|m-n|} \leq \sigma_q \leq \dots \leq \sigma_1$

根据上述定理可知 $\tilde{A}$ 的特征值就是 A 的奇异值, 且正负成对出现.

因此 Lanczos 双对角化算法有限收敛的是 A 的最大奇异值,

而最小奇异值躲到中间了, 我们拿它没有办法.

对于良态问题可对 $A^H A$ 或 $A A^H$ 应用 Lanczos 算法计算最小奇异值, 但效果不太好.

(或者使用课上没讲过的方法: LOBPCG, Jacobi-Davidson)

注意到直接对 $\tilde{A}$ 应用 Lanczos 算法是不保证特征值 (或者说奇异值) 成对收敛的.

我们可以合理地选取初值的形式, 这样会有非零部分在两个分块之间 "跳来跳去" 的现象:

$$\begin{bmatrix} 0 & A \\ A^H & 0 \end{bmatrix} \begin{bmatrix} * \\ * \end{bmatrix} = \begin{bmatrix} * \\ * \end{bmatrix}$$

这可以自动省去一次正交, 并保证 $\tilde{T}_{2k} \in \mathbb{C}^{(2k+1) \times (2k)}$ 的对角线上有精确的零.

给定初值后不断迭代, 可以得到:

(元素的填入参考了后文的 Lanczos 下双对角化算法, 也是邵老师课上讲的版本)

$$\begin{aligned} \tilde{A}Q_{2k} &= \begin{bmatrix} 0 & A \\ A^H & 0 \end{bmatrix} \begin{bmatrix} u_1 & 0_m & u_2 & 0_m & \dots & u_k & 0_m \\ 0_n & v_1 & 0_n & v_2 & \dots & 0_n & v_k \end{bmatrix} \\ &= \begin{bmatrix} 0_m & Av_1 & 0_m & Av_2 & \dots & 0_m & Av_k \\ A^H u_1 & 0_n & A^H u_2 & 0_n & \dots & A^H u_k & 0_n \end{bmatrix} \quad (\text{Lanczos: } \begin{cases} A^H u_k = \beta_{k-1} v_{k-1} + \alpha_k v_k \\ Av_k = \alpha_k u_k + \beta_k u_{k+1} \end{cases} \quad (k = 1, \dots, n)) \\ &= \begin{bmatrix} 0_m & \alpha_1 u_1 + \beta_1 u_2 & 0_m & \alpha_2 u_2 + \beta_2 u_3 & \dots & 0_m & \alpha_k u_k + \beta_k u_{k+1} \\ \alpha_1 v_1 & 0_n & \beta_1 v_1 + \alpha_2 v_2 & 0_n & \dots & \beta_{k-1} v_{k-1} + \alpha_k v_k & 0_n \end{bmatrix} \\ &= \begin{bmatrix} u_1 & 0_m & u_2 & 0_m & \dots & u_k & 0_m & u_{k+1} \\ 0_n & v_1 & 0_n & v_2 & \dots & 0_n & v_k & 0_n \end{bmatrix} \begin{bmatrix} 0 & \alpha_1 & & & & & \\ \alpha_1 & 0 & \beta_1 & & & & \\ & \beta_1 & 0 & \alpha_2 & & & \\ & & \alpha_2 & 0 & \beta_2 & & \\ & & & \beta_2 & 0 & \ddots & \\ & & & & \ddots & \ddots & \beta_{k-1} \\ & & & & & \beta_{k-1} & 0 & \alpha_k \\ & & & & & & \alpha_k & 0 \\ & & & & & & & \beta_k \end{bmatrix} \\ &= Q_{2k+1} \tilde{T}_{2k} \end{aligned}$$

一个有趣的事实: 偶数阶的对角元为零的 Hermite 三对角阵的特征值是正负成对出现的.

(奇数阶的情况只是多了一个零特征值而已)

这是因为形如 T_{2k} 的矩阵可以经过重排变为两个双对角阵构成的分块反序矩阵:

([Properties of zero-diagonal symmetric matrices - Mathematics Stack Exchange](https://math.stackexchange.com/questions/1111111/properties-of-zero-diagonal-symmetric-matrices)).

$$T_{2k} = \begin{bmatrix} 0 & \alpha_1 & & & & \\ \alpha_1 & 0 & \beta_1 & & & \\ & \beta_1 & 0 & \alpha_2 & & \\ & & \alpha_2 & 0 & \beta_2 & \\ & & & \beta_2 & 0 & \ddots \\ & & & & \ddots & \ddots & \beta_{k-1} \\ & & & & & \beta_{k-1} & 0 & \alpha_k \\ & & & & & & \alpha_k & 0 \end{bmatrix}$$

$$P_{2k}^T T_{2k} P_{2k} = \left[\begin{array}{c|c} & \begin{matrix} \alpha_1 & \beta_1 & & & \\ & \alpha_2 & \ddots & & \\ & & \ddots & \beta_{k-1} & \\ & & & \alpha_k \end{matrix} \\ \hline \begin{matrix} \alpha_1 \\ \beta_1 & \alpha_2 \\ & \ddots & \ddots \\ & & \beta_{k-1} & \alpha_k \end{matrix} & \end{array} \right] = \begin{bmatrix} B_k & B_k^H \end{bmatrix}$$

相应地, $\tilde{T}_{2k}$ 可以重排为:

$$\tilde{T}_{2k} = \left[\begin{array}{c|c} \begin{matrix} 0 & \alpha_1 & & & \\ \alpha_1 & 0 & \beta_1 & & \\ & \beta_1 & 0 & \alpha_2 & \\ & & \alpha_2 & 0 & \beta_2 \\ & & & \beta_2 & 0 & \ddots \\ & & & & \ddots & \ddots & \beta_{k-1} \\ & & & & & \beta_{k-1} & 0 & \alpha_k \\ & & & & & & \alpha_k & 0 \end{matrix} & \\ \hline & \beta_k \end{array} \right]$$

$$\tilde{P}_{2k}^T \tilde{T}_{2k} \tilde{P}_{2k} = \left[\begin{array}{c|c} & \begin{matrix} \alpha_1 & \beta_1 & & & \\ & \alpha_2 & \ddots & & \\ & & \ddots & \beta_{k-1} & \\ & & & \alpha_k \end{matrix} \\ \hline \begin{matrix} \alpha_1 \\ \beta_1 & \alpha_2 \\ & \ddots & \ddots \\ & & \beta_{k-1} & \alpha_k \\ & & & \beta_k \end{matrix} & \end{array} \right] = \begin{bmatrix} \tilde{B}_k & B_k^H \end{bmatrix}$$

4.4.7 应用于最小二乘

双对角化可用于求解最小二乘问题 $\min \|Ax - b\|_2$

设小型稠密的列满秩矩阵 $A \in \mathbb{C}^{m \times n}$ ($m \geq n$) 的二对角化为:

$$[U, U_\perp]^H A V = \begin{bmatrix} B \\ 0_{(m-n) \times n} \end{bmatrix}$$

$$\Leftrightarrow U^H A V = B$$

其中 $B \in \mathbb{C}^{n \times n}$ 无论是上双对角阵还是下双对角阵都是可以的.

$$\begin{aligned}
\min_{x \in \mathbb{C}^n} \|Ax - b\|_2 &= \min_{x \in \mathbb{C}^n} \left\| [U, U_\perp] \begin{bmatrix} B \\ 0_{(m-n) \times n} \end{bmatrix} Vx - b \right\|_2 \\
&= \min_{x \in \mathbb{C}^n} \left\| \begin{bmatrix} B \\ 0_{(m-n) \times n} \end{bmatrix} Vx - [U, U_\perp]^H b \right\|_2 \\
&= \min_{x \in \mathbb{C}^n} \left\| \begin{bmatrix} BVx - U^H b \\ -U_\perp^H b \end{bmatrix} \right\|_2 \\
&= \|U_\perp^H b\|_2
\end{aligned}$$

最小二乘解 x_* 满足 $BVx_* = U^H b$, 其求解步骤如下:

- ① 求解双对角方程组 $By = U^H b$ 得到 y_*
 若 B 是上双对角的, 则使用回代法;
 若 B 是下双对角的, 则使用前代法;
 (或者使用 Givens 变换将 B 的次对角元都消成 0, 得到一个上双对角阵进行求解.)
- ② 计算 $x_* = V^H y_*$ (注意 $V \in \mathbb{C}^{n \times n}$ 是酉矩阵)

当 $A \in \mathbb{C}^{m \times n}$ 为大型稀疏矩阵时, 使用 Lanczos 上双对角化是行不通的,

因为这要求我们把整个 $V \in \mathbb{C}^{n \times n}$ 都算出来 (**存疑**)

而使用 Lanczos 下双对角化是可行的, 因为我们迭代过程中我们就可用 Givens 变换求解子问题, 就像 GMRES 那样.

设 $[U, U_\perp]^H A V = \begin{bmatrix} B \\ 0_{(m-n) \times n} \end{bmatrix}$ 是矩阵 $A \in \mathbb{C}^{m \times n}$ ($m \geq n$) 的下双对角化:

$$\begin{aligned}
U &= [u_1, \dots, u_n] \in \mathbb{C}^{m \times n} \text{ such that } U^T U = I_n \\
U_\perp &= [u_{n+1}, \dots, u_m] \in \mathbb{C}^{m \times (m-n)} \text{ such that } U_\perp^T U_\perp = I_{m-n} \\
V &= [v_1, \dots, v_n] \in \mathbb{C}^{n \times n} \text{ such that } V^T V = I_n \\
B &= \begin{bmatrix} \alpha_1 & & & & \\ \beta_1 & \alpha_2 & & & \\ & \beta_2 & \ddots & & \\ & & \ddots & \alpha_{n-1} & \\ & & & \beta_{n-1} & \alpha_n \end{bmatrix}
\end{aligned}$$

其精简形式为 $U^H A V = B$

其中 $\alpha_1, \dots, \alpha_n$ 和 $\beta_1, \dots, \beta_{n-1}$ 均为实数.

和 Lanczos 上双对角化一样, 我们比较下列矩阵方程的列:

$$\begin{aligned}
[A^H u_1, A^H u_2, \dots, A^H u_n] &= A^H U = V B^H = [\alpha_1 v_1, \beta_1 v_1 + \alpha_2 v_2, \dots, \beta_{n-1} v_{n-1} + \alpha_n v_n] \\
[A v_1, \dots, A v_{n-1}, A v_n] &= A V = U B = [\alpha_1 u_1 + \beta_1 u_2, \dots, \alpha_{n-1} u_{n-1} + \beta_{n-1} u_n, \alpha_n u_n]
\end{aligned}$$

定义 $\beta_0 u_0 = 0$ 和 $\beta_n v_{n+1} = 0$, 则我们有如下迭代格式:

(与 Lanczos 上双对角化相比, Lanczos 下双对角化就相当于将 A, A^H 和 u, v 以及 B, B^H 的记号互换了一样)

$$\begin{cases} A^H u_k = \beta_{k-1} v_{k-1} + \alpha_k v_k \\ A v_k = \alpha_k u_k + \beta_k u_{k+1} \end{cases} \quad (k = 1, \dots, n)$$

我们记 $\begin{cases} r_k = A^H u_k - \beta_{k-1} v_{k-1} = \alpha_k v_k \\ p_k = A v_k - \alpha_k u_k = \beta_k u_{k+1} \end{cases}$

根据 U, V 的列标准正交性可知:

$$\begin{cases} \alpha_k = \pm \|r_k\|_2 \\ v_k = \frac{r_k}{\alpha_k} \\ \beta_k = \pm \|p_k\|_2 \\ u_k = \frac{p_k}{\beta_k} \end{cases}$$

恰当地安排它们的顺序, 就得到如下算法:

(Lanczos 下双对角化算法 (Paige-Saunders), Matrix Computation 9.3.4 节)

Given $A \in \mathbb{C}^{m \times n}$ and $u_1 \in \mathbb{C}^n$ such that $\|u_1\|_2 = 1$
 $\beta_0 = 1; v_0 = 0_n; p_0 = u_1;$
 $k = 0;$
while $\beta_k \neq 0$
 $u_k = \frac{p_k}{\beta_k}$
 $k = k + 1$
 $r_k = A^H u_k - \beta_{k-1} v_{k-1}$
 $\alpha_k = \|r_k\|_2$
 $v_k = \frac{r_k}{\alpha_k}$
 $p_k = A v_k - \alpha_k u_k$
 $\beta_k = \|p_k\|_2$
end

其迭代格式可以等价地表示为:

$$A^H U_k = [A^H u_1, \dots, A^H u_k] = [v_1, \dots, v_k] \begin{bmatrix} \alpha_1 & \beta_1 & & \\ & \alpha_3 & \ddots & \\ & & \ddots & \beta_{k-1} \\ & & & \alpha_k \end{bmatrix} = V_k B_k^H$$

$$A V_k = [A v_1, \dots, A v_k] = [u_1, \dots, u_k, u_{k+1}] \begin{bmatrix} \alpha_1 & & & \\ \beta_1 & \alpha_3 & & \\ & \ddots & \ddots & \\ & & \beta_{k-1} & \alpha_k \\ \hline & & & \beta_k \end{bmatrix} = U_{k+1} \tilde{B}_k$$

上述算法以 u_1 为初始向量 (作用在 A^H 上开启迭代), 且新的 β_k, u_{k+1} 是由 A 产生的.

现考虑大型稀疏的最小二乘问题:

$$\min_{x \in \mathbb{C}^n} \|Ax - b\|_2$$

给定初始解 $x_0 \in \mathbb{C}^n$, 记 $r_0 := b - Ax_0$

以 $u_1 := \frac{r_0}{\|r_0\|_2}$ 为初值开始 Lanczos 下双对角迭代.

设第 k 步 Lanczos 下双对角迭代得到了 $AV_k = U_{k+1} \tilde{B}_k = U_k B_k + \beta_k u_{k+1}$

其中 $V_k = [v_1, \dots, v_k] \in \mathbb{C}^{n \times k}$ 和 $U_{k+1} = [u_1, \dots, u_k, u_{k+1}] \in \mathbb{C}^{m \times (k+1)}$ 列标准正交.

而 $\tilde{B}_k \in \mathbb{C}^{(k+1) \times k}$ 为下双对角阵:

$$\tilde{B}_k = \begin{bmatrix} \alpha_1 & & & \\ \beta_1 & \alpha_3 & & \\ & \ddots & \ddots & \\ & & \beta_{k-1} & \alpha_k \\ \hline & & & \beta_k \end{bmatrix}$$

我们的目标是让 $\|Ax - b\|_2$ 在 $x_0 + \text{span}\{V_k\}$ 上达到最小:

$$\begin{aligned} \min_{x \in x_0 + \text{span}\{V_k\}} \|Ax - b\|_2 &= \min_{\{x = x_0 + V_k y : y \in \mathbb{C}^k\}} \|Ax - b\|_2 \\ &= \min_{y \in \mathbb{C}^k} \|A(x_0 + V_k y) - b\|_2 \\ &= \min_{y \in \mathbb{C}^k} \|AV_k y - (b - Ax_0)\|_2 \quad (\text{note that } r_0 = b - Ax_0 = \|r_0\|_2 u_1 \text{ and } AV_k = U_{k+1} \tilde{B}_k) \\ &= \min_{y \in \mathbb{C}^k} \|U_{k+1} \tilde{B}_k y - \|r_0\|_2 u_1\|_2 \\ &= \min_{y \in \mathbb{C}^k} \|\tilde{B}_k y - \|r_0\|_2 \cdot U_{k+1}^H u_1\|_2 \quad (\text{note that } U_{k+1}^H u_1 = e_1) \\ &= \min_{y \in \mathbb{C}^k} \|\tilde{B}_k y - \|r_0\|_2 \cdot e_1\|_2 \end{aligned}$$

这样我们就得到了子问题的具体形式.
其中 e_1 是 $\mathbb{C}^k$ 的第 1 个标准单位基向量.

受 GMRES 的启发, 我们使用 k 个 Givens 变换将下双对角阵 $\tilde{B}_k \in \mathbb{C}^{(k+1) \times k}$ 变为上双对角阵 $\begin{bmatrix} R_k \\ 0_k^T \end{bmatrix}$

对应地, $\|r_0\|_2 e_1$ 在 k 个 Givens 变换的作用下变成一个满的 $k+1$ 维向量.

以 $k=4$ 的情况为例:

$$\begin{aligned}
\tilde{B}_4 &= \begin{bmatrix} * & & & & \\ * & * & & & \\ & * & * & & \\ & & * & * & \\ \hline & & & * & * \\ & & & & * \end{bmatrix} \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \end{bmatrix} = \begin{bmatrix} \|r_0\|_2 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \|r_0\|_2 e_1 \\
G_{1,2} \tilde{B}_4 &= \begin{bmatrix} * & * & & & \\ \boxed{0} & * & & & \\ & * & * & & \\ & & * & * & \\ \hline & & & * & * \\ & & & & * \end{bmatrix} \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \end{bmatrix} = \begin{bmatrix} * \\ \boxed{*} \\ 0 \\ 0 \\ 0 \end{bmatrix} = G_{1,2}(\|r_0\|_2 e_1) \\
G_{2,3}(G_{1,2} \tilde{B}_4) &= \begin{bmatrix} * & * & & & \\ 0 & * & * & & \\ & \boxed{0} & * & & \\ & & * & * & \\ \hline & & & * & * \\ & & & & * \end{bmatrix} \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \end{bmatrix} = \begin{bmatrix} * \\ * \\ \boxed{*} \\ 0 \\ 0 \end{bmatrix} = G_{2,3}(G_{1,2} \|r_0\|_2 e_1) \\
G_{3,4}(G_{2,3} G_{1,2} \tilde{B}_4) &= \begin{bmatrix} * & * & & & \\ 0 & * & * & & \\ & 0 & * & * & \\ & & \boxed{0} & * & \\ \hline & & & * & * \\ & & & & * \end{bmatrix} \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \end{bmatrix} = \begin{bmatrix} * \\ * \\ * \\ \boxed{*} \\ 0 \end{bmatrix} = G_{3,4}(G_{2,3} G_{1,2} \|r_0\|_2 e_1) \\
G_{4,5}(G_{3,4} G_{2,3} G_{1,2} \tilde{B}_4) &= \begin{bmatrix} * & * & & & \\ 0 & * & * & & \\ & 0 & * & * & \\ & & 0 & * & \\ \hline & & & \boxed{0} & * \end{bmatrix} \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \end{bmatrix} = \begin{bmatrix} * \\ * \\ * \\ * \\ \boxed{*} \end{bmatrix} = G_{4,5}(G_{3,4} G_{2,3} G_{1,2} \|r_0\|_2 e_1)
\end{aligned}$$

最后一维上的残差我们是管不了的, 我们只能求解上方的 k 维上双对角方程组.

使用回代法解得 $y_k \in \mathbb{C}^k$, 进而得到第 k 次迭代的近似解 $x_k = x_0 + V_k y_k$

4.4.8 应用于双线性型

给定大型稀疏矩阵 $A \in \mathbb{C}^{n \times n}$ 、解析函数 $f(\cdot)$ 和向量 $u, v \in \mathbb{C}^n$

假设矩阵函数 $f(A)$ 是定义良好的.

- ① 首先考虑实现矩阵函数乘向量: $v \mapsto f(A)v$
设置 $q_1 = \frac{v}{\|v\|_2}$ 并对 A 应用 Lanczos 算法.
设第 k 步 Lanczos 迭代得到 $AQ_k = Q_{k+1}\tilde{T}_k = Q_k T_k + \beta_k q_{k+1}$
我们认为 $AQ_k \approx Q_k T_k$ 近似成立, 则有:
(尽管随着幂次提高而越来越不靠谱)

$$\begin{aligned}
AQ_k &\approx Q_k T_k \\
A^2 Q_k &= A(AQ_k) \approx A(Q_k T_k) \approx (Q_k T_k) T_k = Q_k T_k^2 \\
&\dots \\
A^m Q_k &= A(A^{m-1} Q_k) \approx A(Q_k T_k^{m-1}) = (Q_k T_k) T_k^{m-1} = Q_k T_k^m
\end{aligned}$$

考虑到矩阵函数 $f(A)$ 是 A 的幂级数, 于是我们认为 $f(A)Q_k \approx Q_k f(T_k)$ 近似成立.

对于小型稠密的对称三对角阵 T_k , 我们有很多方法计算其矩阵函数 $f(T_k)$

例如用 QR 算法计算其谱分解 $T_k = U_k \Theta_k U_k^H$, 则 $f(T_k) = U_k f(\Theta_k) U_k^H$

于是我们有:

$$\begin{aligned}
f(A)v &= f(A) \cdot (\|v\|_2 Q_k e_1) \quad (\text{note that } Q_k e_1 = q_1 = \frac{v}{\|v\|_2}) \\
&= \|v\|_2 \cdot f(A) Q_k e_1 \quad (\text{note that } f(A) Q_k \approx Q_k f(T_k)) \\
&\approx \|v\|_2 \cdot Q_k f(T_k) e_1 \\
&= \|v\|_2 \cdot Q_k \cdot [f(T_k) e_1]
\end{aligned}$$

在上述过程中，我们需要保留 Lanczos 向量 $Q_k = [q_1, \dots, q_k]$

- ② 其次考虑实现 Hermite 二次型的计算: $v \mapsto v^H A v$

设置 $q_1 = \frac{v}{\|v\|_2}$ 并对 A 应用 Lanczos 算法.

设第 k 步 Lanczos 迭代得到 $AQ_k = Q_{k+1} \tilde{T}_k = Q_k T_k + \beta_k q_{k+1}$

根据之前的结论，我们认为 $f(A)Q_k \approx Q_k f(T_k)$ 近似成立，且 $f(T_k)$ 已经算出.

于是我们有:

$$\begin{aligned}
v^H f(A)v &= (\|v\|_2 Q_k e_1)^H \cdot f(A) \cdot (\|v\|_2 Q_k e_1) \quad (\text{note that } Q_k e_1 = q_1 = \frac{v}{\|v\|_2}) \\
&= \|v\|_2 \cdot e_1^H Q_k^H f(A) Q_k e_1 \quad (\text{note that } f(A) Q_k \approx Q_k f(T_k)) \\
&= \|v\|_2 \cdot e_1^H f(T_k) e_1
\end{aligned}$$

在上述过程中，我们不需要保留 Lanczos 向量 $Q_k = [q_1, \dots, q_k]$

只需迭代得到 T_k ，计算矩阵函数 $f(T_k)$ ，并取其 $(1, 1)$ 位置上的元素 $e_1^H f(T_k) e_1$ 即可.

- ③ 最后考虑实现双线性型的计算: $u, v \mapsto u^H f(A)v$

复数域上的极化恒等式:

$$\langle x, y \rangle = \frac{1}{4} (\|x + y\|^2 - \|x - y\|^2 + i\|x + iy\|^2 - i\|x - iy\|^2)$$

实数域上的极化恒等式:

$$\langle x, y \rangle = \frac{1}{4} (\|x + y\|^2 - \|x - y\|^2)$$

根据极化恒等式我们有:

(由于这是形式上的内积，故无需假设 $f(A)$ Hermite 正定)

$$u^H f(A)v = \frac{1}{4} [(u+v)^H f(A)(u+v) - (u-v)^H f(A)(u-v) + i(u+iv)^H f(A)(u+iv) - i(u-iv)^H f(A)(u-iv)]$$

于是计算双线性型 $u^H f(A)v$ 的任务就归结为计算 4 个 Hermite 二次型.

我们分别取 $q_1 = \frac{u+v}{\|u+v\|_2}$, $q_1 = \frac{u-v}{\|u-v\|_2}$, $q_1 = \frac{u+iv}{\|u+iv\|_2}$ 和 $q_1 = \frac{u-iv}{\|u-iv\|_2}$ 并应用 Lanczos 算法即可.

(应用举例)

考虑连续时间的一阶线性时不变系统:

$$\begin{cases} x'(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases}$$

- $A \in \mathbb{C}^{n \times n}$ 代表系统矩阵，决定了状态向量 $x(t) \in \mathbb{C}^n$ 之间的动态关系.
- $B \in \mathbb{C}^{n \times m}$ 代表输入矩阵，描述输入信号 $u(t) \in \mathbb{C}^m$ 如何作用于状态向量 $x(t) \in \mathbb{C}^n$
- $C \in \mathbb{C}^{p \times n}$ 代表输出矩阵，决定状态向量 $x(t) \in \mathbb{C}^n$ 如何影响输出信号 $y(t) \in \mathbb{C}^p$
- $D \in \mathbb{C}^{p \times m}$ 代表直接传递矩阵，表示输入信号 $u(t) \in \mathbb{C}^m$ 对输出信号 $y(t) \in \mathbb{C}^p$ 的直接影响

其解为:

$$\begin{cases} x(t) = e^{tA}x(0) + \int_0^t e^{(t-s)A}Bu(s)ds \\ y(t) = Cx(t) + Du(t) \end{cases}$$

当 A 为大型稀疏矩阵时， $e^{tA}x(0)$ 和 $\int_0^t e^{(t-s)A}Bu(s)ds$ 的计算都需要用到 Lanczos 算法.

(其中积分 $\int_0^t e^{(t-s)A}Bu(s)ds$ 我们是通过离散化求和取极限的方式来逼近的)

4.4.9 预反幂法 (PINVIT)

4.4.10 Davidson 算法

4.4.11 FEAST 算法

4.4.12 总结

The End